

ABSTRACT

ExamAI is a full-stack, AI-powered exam-preparation platform designed to connect students with teacher- approved learning material. The platform combines retrieval-augmented generation (RAG) chat, teacher- authored shared quizzes, AI-assisted quiz generation, and student-owned flashcard decks in one web application. It provides separate workflows for students and teachers while keeping authorization, material ownership, and academic comparability explicit in the system design. Students can select a subject, choose the approved materials that should scope a study session, ask questions, receive answers with teacher and filename attribution, attempt published quizzes, review weak topics, and generate flashcards. Teachers can upload PDF, PPTX, and DOCX materials, monitor ingestion status, author or generate quizzes, publish shared assessments, and review class-wide analytics. The backend uses FastAPI, SQLAlchemy, Alembic, Supabase Postgres and Auth, Gemini, Qdrant Cloud, and local sentence-transformer embeddings. The frontend uses React, Vite, React Router, Zustand, Tailwind CSS, and reusable interface primitives. The project demonstrates secure role-based access, metadata-filtered retrieval, concurrent ingestion protection, structured-output retry handling, server-side quiz grading, and measured frontend performance. The current implementation contains 82 offline backend tests and 63 frontend tests. The report presents the motivation, objectives, literature and technology overview, project management approach, requirements, system analysis, testing, system design, results, limitations, future enhancements, and conclusion of the ExamAI project.

TABLE OF CONTENT

Title	
CHAPTER 1 INTRODUCTION	1
1.1 Project Summary	1
1.2 Project Purpose	1
1.3 Project Scope	2
1.4 Objectives	2
1.4.1 Main Objectives	2
1.4.2 Secondary Objectives	3
1.5 Technology and Literature Overview	3
1.5.1 Retrieval-Augmented Generation	3
1.5.2 Vector Search and Embeddings	3
1.5.3 Programming Languages	3
1.5.4 Libraries and Tools	4
1.5.5 Literature Overview	4
1.6 Synopsis	4
CHAPTER 2 LITERATURE SURVEY	5
2.1 Introduction of Survey	5
2.2 Why Survey?	5
2.3 Existing System vs Proposed System	5
CHAPTER 3 PROJECT MANAGEMENT	7
3.1 Project Planning Objectives	7
3.2 Project Scheduling	7
3.2.1 Basic Principle	7
3.2.2 Compartmentalization	7
3.2.3 Work Breakdown Structure	7
3.2.4 Project Organization	8
3.2.5 Timeline Chart	8
3.2.5.1 Time Allocation	9
3.3 Risk Management	9
CHAPTER 4 SYSTEM REQUIREMENTS	11
4.1 User Characteristics	11
4.2 Functional Requirement	11
4.3 Activity and Proposed System	12
4.4 Non Functional Requirement	12
4.5 Hardware and Software Requirement	12
CHAPTER 5 SYSTEM ANALYSIS	14
5.1 Study of Current System	14
5.2 Problems in Current System	14
5.3 Requirement of New System	14
5.4 Process Model	15
5.5 Feasibility Study	15
5.5.1 Technical Feasibility	15
5.5.2 Operational Feasibility	15
5.5.3 Economical Feasibility	15

5.5.4 Schedule Feasibility	15
5.6 Features of New System	16
CHAPTER 6 TESTING	17
6.1 Black-Box Testing	17
6.2 White-Box Testing	17
6.3 Test Cases	18
CHAPTER 7 SYSTEM DESIGN	19
7.1 System Architecture	19
7.2 Class Diagram	20
7.3 Use-Case Diagram	20
7.4 Sequence Diagram	21
7.5 Activity Diagram	22
7.6 Data Flow Diagram (DFD)	23
7.6.1 Level 0 DFD	23
7.6.2 Level 1 DFD	24
7.7 Data Model	25
CHAPTER 8 RESULT AND DISCUSSION	26
8.1 Login and Role-Based Access	26
8.2 Student Dashboard and Subject View	27
8.3 RAG Chat with Material Scope and Citations	28
8.4 Quiz Taking and Feedback	29
8.5 Flashcard Generation and Study	31
8.6 Teacher Material Management	33
8.7 Teacher Quiz Authoring and Publishing	34
8.8 Teacher Analytics and Student Progress	35
8.9 Security and Reliability Results	37
8.10 Overall System Performance	37
CHAPTER 9 LIMITATION AND FUTURE ENHANCEMENT	38
9.1 Limitation	38
9.2 Future Enhancement	38
CHAPTER 10 CONCLUSION	39
10.1 Conclusion	39
CHAPTER 11 APPENDICES	40
11.1 Business Model	40
11.2 Product Deployment Detail	40
11.3 API and Web Service Details	40

LIST OF FIGURES

Figure No	Title	Page No.
Figure 7.1	System architecture	19
Figure 7.2	Class diagram	20
Figure 7.3	Use-case diagram	21
Figure 7.4	Sequence diagram — RAG chat	22
Figure 7.5	Activity diagram — material ingestion	23
Figure 7.6.1	Data flow diagram — Level 0	24
Figure 7.6.2	Data flow diagram — Level 1	25
Figure 8.1	Sign-in form (seeded login, no signup)	27
Figure 8.2	Student dashboard — hero, quick stats, enrolled subjects	28
Figure 8.3	Subject resources grouped by teacher	28
Figure 8.4	RAG chat — scope panel, citations, grounded refusal	29
Figure 8.5	Published quiz list with scores	30
Figure 8.6	Quiz taking view	30
Figure 8.7	Result card — grade, areas to review	31
Figure 8.8	Question review — correct vs wrong	31
Figure 8.9	Flashcard deck list with mastery progress	32
Figure 8.10	Flip-study view	33
Figure 8.11	Teacher materials with ingestion states	34
Figure 8.12	Teacher quiz list — draft/published, manual/AI	35
Figure 8.13	Per-quiz analytics — heatmap, distribution, weak topics	36
Figure 8.14	Student progress roster with at-risk flags	36
Figure 8.15	Teacher dashboard overview	37

LIST OF TABLES

Table No	Title	Page No.
Table 2.3	Existing systems vs ExamAI	6
Table 3.2.5.1	Time allocation	9
Table 3.3	Risk management	10
Table 4.2	Functional requirements	12
Table 4.4	Non-functional requirements	12

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DOCX	Word Open XML Document
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object Relational Mapping
PDF	Portable Document Format
PPTX	PowerPoint Open XML Presentation
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UX	User Experience
UUID	Universally Unique Identifier

CHAPTER 1 INTRODUCTION

- PROJECT SUMMARY
- PROJECT PURPOSE
- PROJECT SCOPE
- OBJECTIVES
- TECHNOLOGY AND LITERATURE OVERVIEW
- SYNOPSIS

1.1 Project Summary

ExamAI is a web application built to make exam preparation more trustworthy and less fragmented for students, while giving teachers control over the material that AI-assisted study is grounded in. Today a student preparing for an examination typically searches through several files, asks a general-purpose chatbot that cannot say where its answer came from, practices from quiz links that differ from student to student, and revises from self-made notes. A teacher, meanwhile, distributes material in one place, collects quiz responses in another, and interprets class performance largely by hand.

ExamAI brings these activities into a single role-aware platform. A student selects an enrolled subject, chooses which teacher-approved materials should scope a study session, asks questions, and receives answers with numbered citations that identify the teacher, the file, and the page or slide each claim comes from. The same student attempts the same published quiz as every classmate in the subject, receives per-question feedback with weak topics, and generates personal flashcard decks from selected material for revision. A teacher uploads PDF, PPTX and DOCX materials, monitors their processing status, authors quizzes manually or with AI assistance, publishes them for the class, and reviews performance through per-quiz analytics and cross-quiz student progress views.

The application is built using React for the frontend, FastAPI (Python) for the backend, PostgreSQL for relational data, Qdrant for vector retrieval, and Gemini for structured generation. Access control follows subject membership on both sides: teachers act through subject membership, students through subject enrollment.

1.2 Project Purpose

The purpose of ExamAI is to give students one reliable place to prepare for examinations from material their teachers actually approve — and to make every AI-generated answer traceable back to that material. An answer that cannot say which teacher's file and which page it came from leaves a student with no one to ask when in doubt. ExamAI treats attribution as a core requirement, not a cosmetic addition: each citation carries the teacher's name, the filename, and the source page or slide, so a confused student knows exactly who to contact. (Contacting the teacher remains out of scope; the system only surfaces who and what.)

For teachers, the purpose is complementary: a practical way to distribute approved material, create comparable assessments in which every student answers the same questions, and inspect class-wide progress instead of reconciling scattered quiz tools. Building it as one system, rather than four disconnected tools, keeps study activity, assessment, and analytics connected by the same data and authorization rules.

1.3 Project Scope

The system is capable of:

- Secure email/password login for provisioned users, with the role (student or teacher) derived from the backend profile.
- Subject dashboards: enrolled subjects for students, teaching subjects for teachers, with multi-teacher subjects supported.
- Teacher-owned material uploads (PDF, PPTX, DOCX, 25 MB limit) with processing / ready / failed status, retry, and deletion.
- Subject-scoped RAG chat over a per-session material selection, returning answers with numbered teacher/file/page citations.
- Manual and AI-assisted quiz authoring converging on the same question shape, with a draft → review → publish flow.
- Shared published quizzes: all students in a subject take the same quiz; server-side grading with per-question feedback and weak-topic detection.
- Student-owned flashcard deck generation from selected materials, with mastery tracking (new / learning / mastered).
- Teacher analytics: per-quiz views (accuracy heatmap, grade distribution, weak topics) and cross-quiz student progress (averages, completion, at-risk flags, drill-down).

The current version has a few known limits:

- It is a web application only — there is no mobile app yet.
- Users are provisioned by an explicit seed operation; there is no public signup, password reset, OAuth, or admin role.
- There is no in-app teacher–student messaging or broadcast.
- Student-generated graded quizzes are deferred as explicit v2 scope, to protect the fairness of shared assessment.
- Retrieval is dense-vector only; hybrid dense-plus-sparse search is a flagged future enhancement, not part of this delivery.
- The live behaviour of generation features depends on the configured Gemini model, quota and billing, which must be verified per deployment (the configured default is `gemini-2.5-flash`).

1.4 Objectives

1.4.1 Main Objectives

- To build a single role-aware platform for AI-assisted exam preparation.

- To ground every study answer in teacher-approved material and attribute each claim to its teacher, file, and page or slide.
- To keep assessment fair by publishing one shared quiz per topic for all students in a subject.
- To return feedback beyond a total score: per-question results and weak-topic detection.
- To let students generate personal revision decks from their own selected materials.
- To give teachers class-wide comparability through per-quiz analytics and cross-quiz progress views.

1.4.2 Secondary Objectives

- To enforce subject membership and role authorization in service-layer logic, not only in route declarations.
- To preserve source metadata end to end — parsing, chunking, embedding, retrieval, citation — so attribution never depends on a per-request database lookup.
- To handle structured model-output failures through schema-driven, error-aware retries.
- To keep the ingestion pipeline safe under concurrent upload, retry, and delete operations.
- To keep the codebase organized (thin routes, logic in services, schemas separate from models) so new features do not require rewriting old ones.

1.5 Technology and Literature Overview

1.5.1 Retrieval-Augmented Generation

Retrieval-augmented generation is an approach that combines information retrieval with language generation. Instead of asking a model to answer from its general training alone, the system first retrieves relevant passages from a controlled collection and supplies them as context, so the answer is grounded in identifiable sources. ExamAI uses this approach for subject chat and for AI-assisted quiz generation: retrieval is constrained by subject enrollment and the student's selected materials before the vector database is ever queried.

1.5.2 Vector Search and Embeddings

An embedding represents text as a numerical vector, so that semantically related passages can be compared by distance. ExamAI embeds material chunks locally with the `all-MiniLM-L6-v2` sentence model and stores them in a single Qdrant collection shared across all subjects and materials. Each stored point carries payload metadata — subject, material, teacher, filename, teacher name, and a stable source locator (page, slide, or paragraph number) — which makes metadata-filtered retrieval and citation attribution possible without per-chunk database round-trips.

1.5.3 Programming Languages

Programming languages provide the instructions a computer executes to build software systems. ExamAI's backend and ingestion pipeline are written in Python, and its frontend is written in JavaScript (as React/JSX). Python was chosen for the backend because the AI ecosystem — embedding libraries, document parsers, and model clients — is strongest there; JavaScript/React was chosen for the frontend because the interface is a dynamic, stateful study workspace.

1.5.4 Libraries and Tools

The project uses several important libraries:

- React.js — used for building the interactive study interface (dashboards, chat, quiz taking, flashcards, analytics).
- Vite — provides fast builds and the development server for the frontend.
- FastAPI — the Python web framework serving the HTTP API, with automatic request validation and interactive API documentation.
- SQLAlchemy + Alembic — database access with defined schema, plus versioned migrations.
- Pydantic — validates API shapes and doubles as the structured-output contract for model generation.
- Supabase — provides authentication, PostgreSQL hosting, and private file storage.
- Qdrant client — metadata-filtered vector search over embedded material chunks.
- Zustand, React Router, Tailwind CSS — client state, navigation, and styling.

1.5.5 Literature Overview

Digital learning platforms have been studied and built extensively: learning-management systems organize content and grades, MOOC platforms deliver courses at scale, flashcard tools support spaced revision, and general-purpose chatbots answer arbitrary questions fluently. What the survey in Chapter 2 shows is that none of the widely used options combine the four properties an examination setting needs — teacher-approved sources, answers attributed to a teacher and page, one shared assessment for the whole class, and class-wide analytics — in a single system. ExamAI is built specifically to close that gap, as detailed in Chapter 2.

1.6 Synopsis

ExamAI is a web-based exam-preparation platform in which students study from teacher-approved materials through attributable RAG chat, take the same teacher-published quizzes as their classmates, revise with self-generated flashcards, and teachers monitor class performance through analytics. It is built with React, FastAPI, PostgreSQL, Qdrant, and Gemini, with authorization enforced at the service layer and source metadata preserved from ingestion to citation. The following chapters present the literature survey, project management, requirements, analysis, testing, design, results, limitations, and conclusion of the project.

CHAPTER 2 LITERATURE SURVEY

- INTRODUCTION OF SURVEY
- WHY SURVEY?
- EXISTING SYSTEM VS PROPOSED SYSTEM

2.1 Introduction of Survey

Before building ExamAI, widely used learning platforms and study tools were studied to understand what they already do well and where they fall short for examination preparation. The review focused on five popular, representative options that students and teachers actually use: Moodle (the most widely deployed open-source learning-management system), Google Classroom (the most widely used lightweight classroom platform), Quizlet (the most popular flashcard-based revision tool), general-purpose LLM chatbots such as ChatGPT (what students most commonly ask study questions today), and NotebookLM (Google's retrieval-grounded study assistant over user-supplied sources — the closest existing product to our approach).

The comparison was deliberately operational rather than theoretical: it asks what each system does about content ownership, answer traceability, assessment comparability, feedback, analytics, and access control — the six properties Section 1.5.5 identifies as necessary in an examination setting.

2.2 Why Survey?

A survey of existing platforms is useful because it shows what students and teachers can already do elsewhere, so a new project does not simply repeat what already exists. It also exposes real gaps that a new system can be built specifically to close, instead of guessing what users might need. For ExamAI, the survey served three concrete purposes. First, it confirmed that content management (Moodle, Classroom) and revision tools (Quizlet) exist but are disconnected from AI-grounded question answering. Second, it confirmed that students already use generic chatbots for study help — but those answers carry no source attribution, which is precisely the trust problem ExamAI addresses. Third, it positioned the closest alternative, NotebookLM, and showed where a classroom system must go further: teacher ownership, shared assessment, and class analytics, none of which a personal study assistant provides.

2.3 Existing System vs Proposed System

Sr. No.	Aspect	Existing Systems	ExamAI (Proposed)
1	Source control	Moodle / Classroom store teacher files, but AI answers (ChatGPT) are disconnected from them; NotebookLM grounds answers only in the student's own uploads	Teacher-owned, subject-scoped, status-checked materials are the <i>only</i> retrieval corpus

Sr. No.	Aspect	Existing Systems	ExamAI (Proposed)
2	Answer traceability	ChatGPT answers carry no source; Classroom/Moodle have no QA at all; NotebookLM cites the student's documents, not a teacher	Every answer carries numbered citations with teacher name, filename, and page/slide locator
3	Assessment fairness	Quizlet/Forms-style quizzes are typically per-link and incomparable across a class	One published quiz per topic for all students in the subject, enabling class-wide comparison
4	Feedback depth	Scores usually stop at a total; weak-topic analysis is absent	Server-side grading returns per-question feedback plus weak-topic detection per attempt
5	Teacher insight	Scattered gradebooks (Classroom), or nothing (chatbots, Quizlet)	Dedicated per-quiz analytics (heatmap, grade distribution) plus cross-quiz student progress with at-risk flags
6	Access control	Coarse roles; students can paste any content into a chatbot	Service-layer membership checks: enrollment for students, subject membership for teachers, per-request material validation

Table 2.3: Existing systems vs ExamAI.

Three gaps appeared consistently across all five options studied:

- **Attribution gap.** The tools students actually ask questions of (generic chatbots) cannot say which teacher's file and which page an answer came from. The tools that organize teacher files (Moodle, Classroom) cannot answer questions at all.
- **Comparability gap.** Revision and quiz tools are built around individual practice links. Nothing guarantees that every student in a class answers the same questions, so teachers cannot compare performance fairly.
- **Oversight gap.** No surveyed option connects study help, assessment, and class-wide analytics through one authorization model. Teachers either get a gradebook without study context, or study tools without any teacher visibility.

ExamAI is built directly around closing these three gaps: teacher-owned retrieval with teacher-attributed citations, shared published quizzes, and analytics fed by the same attempts — all guarded by subject membership.

CHAPTER 3 PROJECT MANAGEMENT

- PROJECT PLANNING OBJECTIVES
- PROJECT SCHEDULING
- RISK MANAGEMENT

3.1 Project Planning Objectives

- Define clear goals — what the platform must do for students and for teachers — before writing any code.
- Gather requirements from the literature survey, fixing the teacher-owned / shared-assessment / attributable-answer boundaries early.
- Design the system — data model, service boundaries, and API contracts — before building features on top of them.
- Build in dependency order — authentication and data access first, then materials, then ingestion, then retrieval, then quizzes and flashcards, then analytics — so each layer can be checked before the next is built on it.
- Validate risky integrations early (vector retrieval, structured model output, concurrent ingestion) behind service boundaries.
- Test continuously — offline backend and frontend suites run throughout, not only at the end.
- Keep a stable, working build ahead of every review milestone, deferring explicit v2 scope rather than destabilizing the core.

3.2 Project Scheduling

3.2.1 Basic Principle

The project was built in layers, one on top of the other: authentication and the relational data model first, then subject and material management, then the ingestion pipeline, then RAG chat, then quizzes and flashcards, and finally analytics and hardening. Each layer was verified on its own — for example, retrieval filtering was tested against seeded data before any generation feature consumed it — so problems were caught early, one layer at a time.

3.2.2 Compartmentalization

The system was split into clear, separate modules so each part could be built and tested on its own: a data layer (users, subjects, memberships, materials, quizzes, attempts, decks), a backend service layer (ingestion, retrieval, generation, grading, analytics), an API layer of thin route handlers, and a frontend layer (pages, stores, API client). The Pydantic schemas shared between the API and the model-output contracts act as the agreed interfaces between modules. Splitting the work this way meant a bug in one module rarely affected the others, and responsibilities could be divided among team members along module lines (Section 3.2.4).

3.2.3 Work Breakdown Structure

Phase	Primary work	Main outcome
0 — Foundations	Authentication, schema, migrations, project conventions	Seeded login with role derivation; versioned data model
1 — Subject & material access	Roles, enrollment, teacher-owned material management	Membership-gated subject views; upload with status tracking
2 — Ingestion	PDF/PPTX/DOCX parsing, chunking, embeddings, Qdrant upserts	processing → ready / failed pipeline with retry and safe delete
3 — RAG chat	Filtered retrieval, citation mapping, structured generation	Attributable answers scoped to per-session material selection
4 — Quizzes	Manual + AI-assisted authoring, publish flow, server-side grading	Shared published quizzes with feedback and weak topics
5 — Flashcards	Student-owned deck generation, mastery tracking	Personal revision decks from selected materials
6 — Analytics & performance	Per-quiz and cross-quiz read models; frontend optimization	Teacher dashboards; route splitting, caching, prefetch behaviour
7 — Hardening	Auth hardening, concurrency guards, test suites, cleanup	82 offline backend tests; 63 frontend tests; audit fixes

Table 3.1: Work breakdown structure.

3.2.4 Project Organization

- **Internal Guide (Ms. Shreya Bhatt, Assistant Professor, CSE):** guides and reviews the project — scope approval, design and RAG-architecture feedback, milestone reviews, and report evaluation.
- **Patel Swet (IU2341230111) — core development:** system architecture, backend services and integration, AI/RAG pipeline (ingestion, retrieval, citations, generation, retries), data model and API contracts, and overall technical integration.
- **Khatri Keshav (IU2341230068) — frontend support:** study and dashboard interface work — pages, components, loading/empty/error states, and client-side presentation behaviour.
- **Patel Dhairya (IU2341230041) — backend support:** supporting backend functionality outside the core AI/RAG implementation — route/handler assistance, validation, and service-level support tasks.
- **End users:** students (study, quizzes, revision) and teachers (materials, assessments, analytics), whose needs define acceptance.

This allocation reflects the work actually performed: the core AI/RAG architecture, retrieval pipeline, and system integration were built by Swet; Keshav and Dhairya hold contained, viva-explainable responsibilities in the frontend and supporting backend respectively.

3.2.5 Timeline Chart

3.2.5.1 Time Allocation

Development ran over fourteen weeks, from the last week of July 2026 to the last week of October 2026 (repository history: first commit 24 Jul 2026), ahead of the November 2026 submission.

Phase	Duration	Description
Requirement analysis	2 weeks (24 Jul – 6 Aug)	Survey of existing platforms (Chapter 2), scope and role definition.
System design	2 weeks (7 – 20 Aug)	Data model, service boundaries, API contracts, UI layout.
Development	6 weeks (21 Aug – 1 Oct)	Foundations → materials → ingestion → RAG chat → quizzes → flashcards → analytics, in dependency order.
Testing	2 weeks (2 – 15 Oct)	Offline backend suite, frontend suite, manual end-to-end passes, bug fixes.
Hardening & documentation	2 weeks (16 – 29 Oct)	Auth/concurrency review, cleanup, report writing, demo preparation.

Table 3.2.5.1: Time allocation.

3.3 Risk Management

Risk	Description	Mitigation Strategy
LLM model/quota changes	Generation features depend on a live model name, quota and billing.	Verify the live model early per deployment; schema-first prompts with error-aware retries so failures are structured, not silent.
Incorrect retrieval scope	Answers could cite material from a subject the student is not enrolled in.	Validate every material ID in Postgres before querying Qdrant; filter by both subject and material IDs (defence in depth).
Concurrent ingestion	Simultaneous upload / retry / delete could revive deleted rows or corrupt status.	Per-material locks, row-level guards, and version-safe conditional status updates.
Unclear role boundaries	Students reaching teacher functions or other subjects' content.	Service-layer membership checks on every subject-scoped operation; 403 (not 404) for unenrolled access.
Slow external services	Embedding, vector DB, or model latency hurting responsiveness.	Blocking work off the event loop; bounded caching and request deduplication on the client.
Scope creep	Personal quizzes, messaging, mobile apps delaying the core.	Explicit in-scope / future-scope split from Section 1.3, enforced at planning.

Risk	Description	Mitigation Strategy
Time constraints	Any phase overrunning into review milestones.	Stable-build-ahead-of-review rule; risky integrations isolated behind service boundaries.

Table 3.3: Risk management.

CHAPTER 4 SYSTEM REQUIREMENTS

- USER CHARACTERISTICS
- FUNCTIONAL REQUIREMENT
- ACTIVITY AND PROPOSED SYSTEM
- NON FUNCTIONAL REQUIREMENT
- HARDWARE AND SOFTWARE REQUIREMENTS

4.1 User Characteristics

ExamAI has two kinds of users. **Students** can view enrolled subjects, scope study sessions to approved materials, ask questions, attempt published quizzes, review feedback, and generate and study flashcards. **Teachers** can manage subject materials (upload, monitor, retry, delete), author and publish quizzes manually or with AI assistance, and inspect quiz analytics and student progress. No special technical knowledge is needed — the interface is designed to be usable by anyone comfortable browsing a normal website, and no user is assumed to understand vector search or language-model operation.

4.2 Functional Requirement

ID	Requirement
FR-01	The system shall authenticate a provisioned user with email and password and derive the role from the backend profile.
FR-02	The system shall allow teachers to upload PDF, PPTX and DOCX materials for a subject, within the configured size limit.
FR-03	The system shall process uploaded material and expose processing, ready, and failed status with retry and delete.
FR-04	The system shall allow enrolled students to select ready materials to scope a study session.
FR-05	The system shall return RAG answers with numbered citations identifying teacher, file, and source location.
FR-06	The system shall allow teachers to create manual or AI-assisted quiz drafts in a common question shape.
FR-07	The system shall require teacher publication before a quiz becomes available to students.
FR-08	The system shall grade attempts on the server and return per-question feedback with weak topics.
FR-09	The system shall generate personal flashcard decks from student-selected material, with mastery tracking.

ID	Requirement
FR-10	The system shall provide per-quiz analytics and cross-quiz student progress views to teachers.

Table 4.2: Functional requirements.

4.3 Activity and Proposed System

A typical student activity begins with login and subject selection. The student opens the material-scope panel, chooses which ready materials should ground the session, and asks a question; the system returns an answer whose numbered citations identify the teacher and file behind each claim. The student may then attempt a published quiz under its time limit, review per-question feedback and weak topics, and generate a flashcard deck from the same or another material selection for revision.

A typical teacher activity begins with subject access, continues through material upload and ingestion monitoring, then quiz authoring — written manually or drafted by the AI from ready material and edited — followed by publication and analytics review. AI-generated quizzes always remain drafts until a teacher reviews and publishes them, keeping the teacher, not the model, responsible for assessment content. Saving personal study state is the student's domain; publishing shared assessment is the teacher's; the system never mixes the two.

4.4 Non Functional Requirement

Category	Requirement
Security	Role and subject-membership authorization enforced in services; access tokens kept in memory, refresh via HttpOnly cookie; CORS allow-list.
Reliability	Malformed model output, failed ingestion, retry/delete races, and concurrent operations handled without corrupting state.
Performance	Blocking parser/embedding work kept off the event loop; frontend uses route splitting, bounded caching, request deduplication, and stale-response protection.
Maintainability	Thin routes, schemas separate from ORM models, business logic in services; shared contracts via API envelopes.
Usability	Loading, empty, failure, and success states throughout; the active material scope always visible to the student.
Traceability	Every citation retains teacher, file, and source-location metadata from ingestion to answer.

Table 4.4: Non-functional requirements.

4.5 Hardware and Software Requirement

Hardware Requirement. As a web application, ExamAI needs no special hardware — any device that can run a modern browser can use it.

- Processor: any modern dual-core processor or above.
- RAM: 4 GB or more (8 GB recommended for local backend development with local embeddings).
- Internet connection: required to reach the API and its managed services.
- Input/Output: a screen with keyboard/mouse or touchscreen.

Software Requirement.

Layer	Technology / version (as used in this project)
Frontend	React 19, Vite 8, React Router 7, Zustand 5, Tailwind CSS 4
Backend	Python 3.14, FastAPI, Uvicorn, SQLAlchemy 2 + Alembic, Pydantic 2
AI / retrieval	sentence-transformers (all-MiniLM-L6-v2), qdrant-client ≥ 1.16 , Gemini (gemini-2.5-flash configured default — verify per deployment)
Data & files	Supabase Postgres + Auth + private Storage
Tooling	Git & GitHub; FastAPI Swagger UI (/docs); pytest (82 backend tests); Vitest + Testing Library (63 frontend tests)

Server Hosting Requirement. The system is developed and verified as a local prototype and is deployment-ready: the FastAPI backend runs under Uvicorn, the frontend builds to static assets via Vite, and data, authentication, and file storage target managed Supabase (Postgres) with vector search on Qdrant Cloud. The reference production layout is Render.com (backend API) + static frontend hosting + managed Supabase and Qdrant services. The application has not yet been deployed online; deployment remains planned future work.

CHAPTER 5 SYSTEM ANALYSIS

- STUDY OF CURRENT SYSTEM
- PROBLEMS IN CURRENT SYSTEM
- REQUIREMENT OF NEW SYSTEM
- PROCESS MODEL
- FEASIBILITY STUDY
- FEATURES OF NEW SYSTEM

5.1 Study of Current System

The conventional exam-preparation workflow is distributed across unconnected tools. A student reads lecture files locally, searches the web or asks a general chatbot for explanations, practices from whatever quiz links are shared, and revises from personal notes. A teacher distributes material in one location (classroom drive, LMS page), collects quiz responses in another (forms, quiz links), and manually interprets performance. Study activity and assessment share no data: what a student asked about yesterday has no connection to what they are tested on today, and what a class got wrong has no automatic path back to the material that should be re-studied.

5.2 Problems in Current System

- Fragmented sources — explanations live in files, chatbots, and web pages with no single approved corpus.
- Unattributed answers — chatbot responses carry no teacher, file, or page reference, so doubts cannot be resolved with the right teacher.
- No access semantics — copied files and shared links bypass any notion of enrollment or ownership.
- Incomparable assessment — per-link quizzes differ between students, so class-wide comparison is unreliable.
- Shallow feedback — a total score says what was achieved, not which topics are weak.
- Manual revision overhead — flashcards and summaries must be built entirely by hand.

5.3 Requirement of New System

- One approved corpus per subject: teacher-owned, status-checked materials as the only retrieval ground.
- Source-aware answers: every claim attributable to teacher, file, and page/slide.
- Subject-level authorization: enrollment for students, membership for teachers, validated per request.
- Shared assessment: one published quiz per topic for the whole class.
- Feedback with diagnosis: per-question results and weak-topic detection.

- Personal revision content: student-generated decks from selected materials.
- Connected analytics: per-quiz and cross-quiz teacher views fed by the same attempts.

5.4 Process Model

ExamAI was built using an incremental development model: requirements were converted into data and service contracts, then each phase delivered a vertical slice across backend and frontend, tested offline where possible, and reviewed before the next slice began. External-service risks (model availability, vector-database behaviour, embedding cost) were isolated behind utility and service boundaries, so the core logic never depends directly on a vendor response shape. Cleanup and hardening passes followed once features stabilized, rather than interrupting feature development.

5.5 Feasibility Study

5.5.1 Technical Feasibility

The system is technically feasible because each required function maps to a mature, well-documented technology: FastAPI and React for the web boundary, PostgreSQL for relational authorization and assessment data, Qdrant for filtered vector retrieval, and Gemini for structured generation validated by Pydantic schemas. All are available as free or development-tier services at this project's scale, so no specialized infrastructure is needed. The principal uncertainty — live model availability and quota — is contained by configuration and error-aware generation paths.

5.5.2 Operational Feasibility

The workflows match the real responsibilities of the two roles: students see only approved material and their own progress; teachers retain control over source files and shared quizzes, including mandatory review of AI-drafted questions. Most study features need no training beyond normal web use. The system is therefore operationally suitable as a classroom prototype; institutional deployment would additionally require onboarding, support, backup, and privacy procedures.

5.5.3 Economical Feasibility

The prototype uses open-source frameworks and managed free or development-tier services. Local embeddings avoid repeated remote-embedding cost. The variable costs are hosted database, vector storage, file storage, and model usage — each replaceable or scalable independently, since no business logic is locked to a vendor SDK beyond thin utility wrappers.

5.5.4 Schedule Feasibility

The phased plan in Section 3.2.3 is schedule-feasible because it keeps a stable build ahead of review milestones and explicitly defers non-essential features (personal graded quizzes, messaging, mobile clients, hybrid retrieval). Building in dependency order meant later phases consumed stable contracts instead of waiting on open design decisions.

5.6 Features of New System

- Secure login with backend-derived roles; enrollment- and membership-gated subject access.
- Teacher-owned material management: upload, processing/ready/failed states, retry, deletion, multi-teacher visibility.
- Document ingestion for PDF, PPTX, and DOCX with format-aware chunking and source-locator preservation.
- Attributable RAG chat over per-session material selection, with numbered teacher/file/page citations.
- Shared quizzes: manual and AI-assisted authoring, draft → publish flow, server-side grading with weak topics.
- Personal flashcard generation with mastery-state tracking.
- Teacher analytics: question-accuracy heatmap, grade distribution, weak topics per quiz; averages, completion, at-risk flags, and drill-down across quizzes.
- Frontend performance safeguards: code-split routes, bounded caching, request deduplication, stale-response protection.

CHAPTER 6 TESTING

- BLACK-BOX TESTING
- WHITE-BOX TESTING
- TEST CASES

6.1 Black-Box Testing

The following black-box tests check that each feature behaves correctly from a user's point of view, without looking at the underlying code:

- Login Testing — a provisioned student or teacher can log in with the correct email and password and receives the correct role.
- Invalid Login Testing — wrong credentials are rejected with an appropriate error.
- Unauthorized Subject Testing — a student requesting another subject's materials receives a forbidden response, not data.
- Material Upload Testing — a teacher upload of a supported file type within the size limit is accepted and enters processing.
- Invalid Upload Testing — an unsupported file type or oversized file is rejected with a clear message and never ingested.
- Chat Scope Testing — a question scoped to authorized, ready materials returns an answer with numbered teacher/file citations.
- Cross-Subject Chat Testing — a request naming a material ID from another subject is refused before any vector query runs.
- Quiz Publication Testing — a draft quiz is invisible to students until the teacher publishes it.
- Attempt Testing — submitting an attempt returns a score with per-question feedback and weak topics; duplicate submission does not duplicate the score.
- Flashcard Testing — deck generation from selected materials produces reviewable cards with mastery states.
- Analytics Testing — per-quiz and cross-quiz views return their separate read models only to the subject's teachers.

6.2 White-Box Testing

The following checks were made against the internal logic of the system:

- Input Validation Logic — Pydantic schemas reject malformed input (bad identifiers, out-of-range values, wrong file types) before it reaches services or storage.
- Authorization Logic — service-layer membership checks (enrollment for students, subject membership for teachers) verified for both allowed and denied paths.
- Retrieval Filter Logic — the Qdrant filter always carries both subject and material constraints built from Postgres-validated IDs.

- Citation Mapping Logic — each [n] marker in generated text resolves to the payload metadata (teacher, filename, locator) of the retrieved chunk.
- Structured-Output Retry Logic — malformed model JSON (code fences, trailing prose, wrong field types) triggers an error-aware retry carrying the verbatim error, the bad response, and the schema.
- Grading Logic — server-side scoring, per-question correctness, and weak-topic derivation checked against hand-computed attempts.
- Ingestion State Logic — status transitions (processing → ready / failed), retry from failure, and version-safe updates under concurrent operations.
- API Contract Logic — every endpoint exercised with valid and invalid payloads through the interactive API documentation; standardized response envelopes verified.

6.3 Test Cases

Test ID	Test Case	Input	Expected Result
TC1	Valid student login	Correct provisioned email + password	Session established, student role returned
TC2	Student requests another subject's material	Subject ID with no enrollment row	Forbidden response
TC3	Teacher uploads unsupported file	Executable / disallowed extension	Validation error; file not ingested
TC4	Student selects unrelated material ID	Material ID from another subject	Authorization fails before vector query
TC5	RAG answer generated	Question + authorized ready scope	Answer with numbered citations carrying teacher and filename
TC6	Malformed model JSON	Forced bad structured output	Error-aware retry validates the corrected output
TC7	Student submits quiz twice	Same attempt submitted again	Idempotent; score not duplicated
TC8	Teacher reads analytics	Per-quiz and progress requests	Separate correct read models returned
TC9	Frontend stale response	Slow earlier request resolving last	Older response does not overwrite current state

Table 6.1: Representative test cases — all passing.

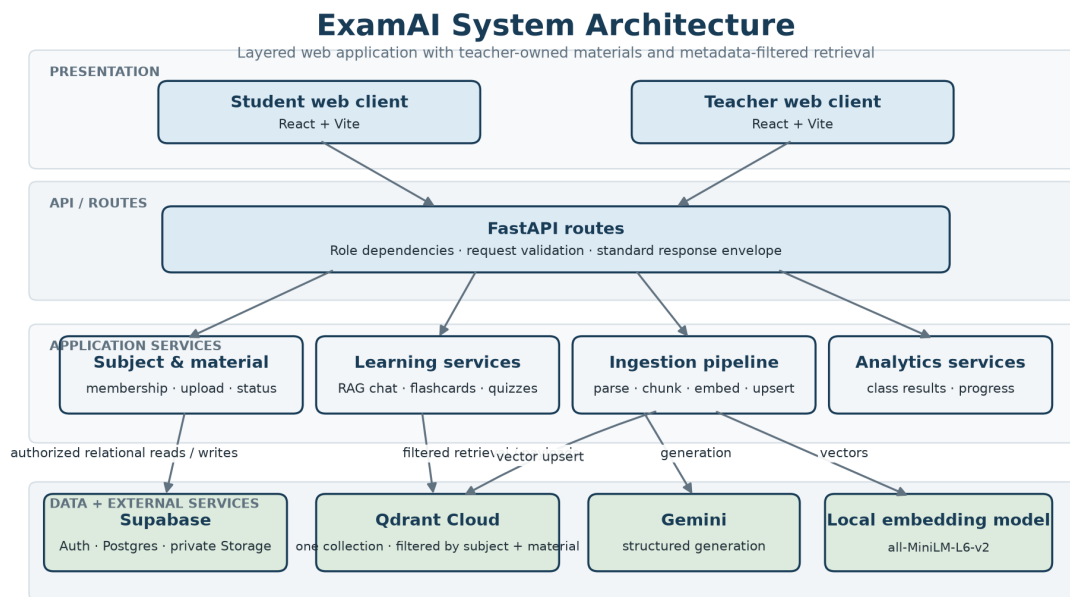
The backend suite (82 tests across smoke, phase, and review-fix files) runs fully offline in under a second; the frontend suite (63 tests across routes, stores, API client, and components) runs under Vitest. Integration with hosted services (Supabase, Qdrant, Gemini) remains environment-dependent and is covered by manual end-to-end passes rather than automated tests.

CHAPTER 7 SYSTEM DESIGN

- SYSTEM ARCHITECTURE
- CLASS DIAGRAM
- USE-CASE DIAGRAM
- SEQUENCE DIAGRAM
- ACTIVITY DIAGRAM
- DATA FLOW DIAGRAM (DFD)
- DATA MODEL

7.1 System Architecture

The architecture has a presentation layer, an API and service layer, a relational data layer, a vector retrieval layer, and external AI and storage services. The frontend calls the FastAPI API directly. Route handlers stay thin: they authenticate the request and delegate to services. Services enforce membership, read or write relational state, and call ingestion, retrieval, generation, or analytics components as needed.



Subject authorization is enforced in services. Material vectors carry material_id, teacher_id and subject_id for scoped retrieval and citation attribution.

Figure 7.1: System architecture

Layer	Responsibilities
Frontend	Pages, routing, stores, API client, caching, loading and error states

Layer	Responsibilities
API	Authentication, request validation, standardized response envelope, request IDs
Services	Materials, ingestion, RAG chat, quizzes, flashcards, analytics; all authorization
Relational data	Users, subjects, memberships, materials, quizzes, attempts, decks
Vector + AI	Qdrant filtered retrieval, Gemini structured generation, local embeddings
Storage	Private teacher material files, application configuration

7.2 Class Diagram

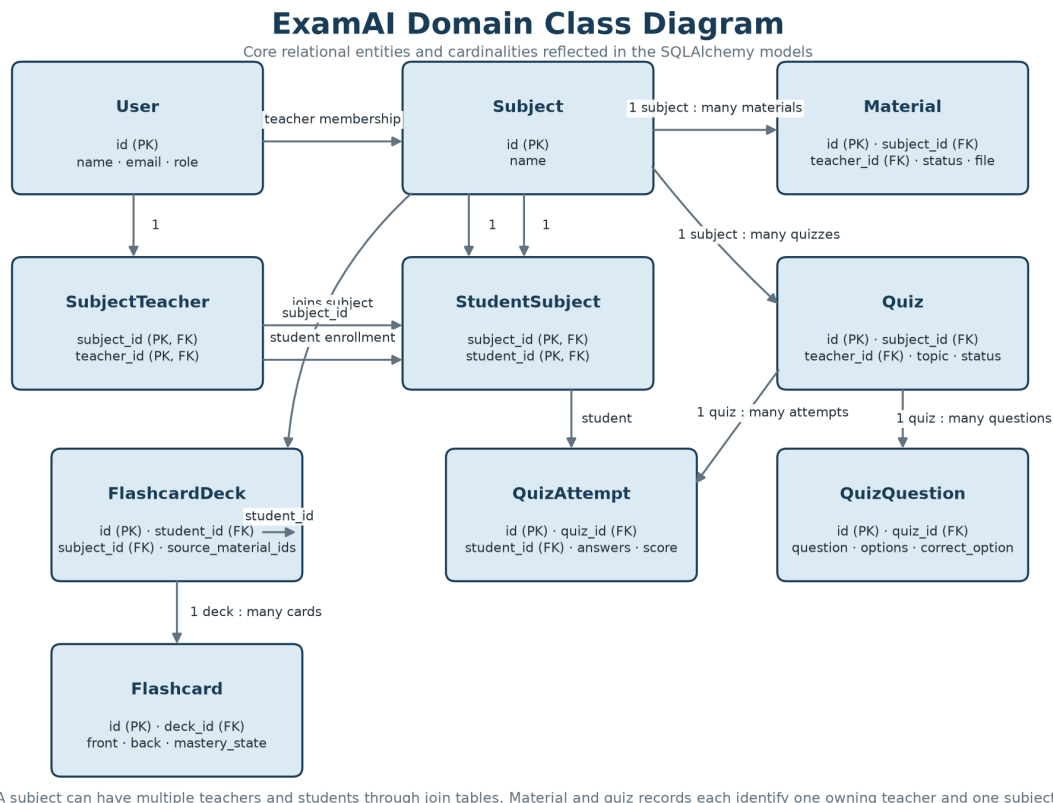


Figure 7.2: Class diagram

The domain centres on Subject: teachers join it through membership, students through enrollment; materials and quizzes belong to exactly one subject and one teacher; attempts belong to one student and one quiz; decks belong to one student and record the source material identifiers used at generation time.

7.3 Use-Case Diagram

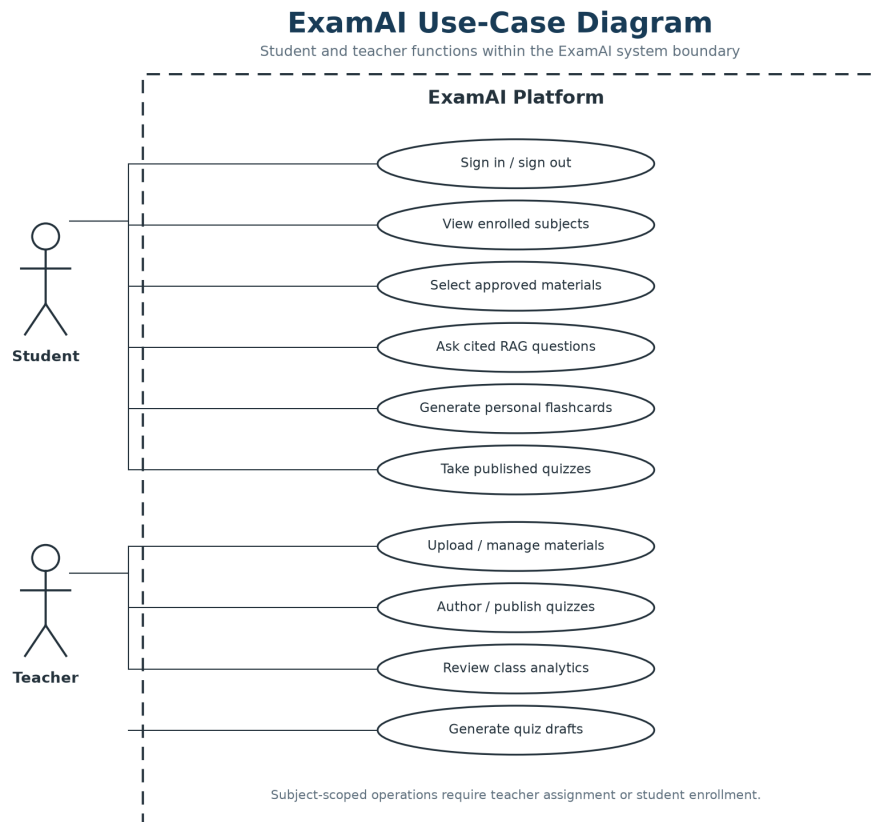


Figure 7.3: Use-case diagram

Both roles authenticate through the same sign-in flow. Subject-scoped requests are authorized against the caller's teacher assignment or student enrollment before protected data is returned or changed.

7.4 Sequence Diagram

The sequence below shows one representative interaction — RAG chat — step by step, since this flow touches every layer of the system in a single user action: the frontend sends subject, selected material identifiers, and a question; the service checks enrollment and validates every identifier in Postgres; the question is embedded; Qdrant is queried with subject-plus-material filters; Gemini receives numbered context; the response is parsed; citation markers are mapped to payload metadata; and the API returns answer text plus citations.

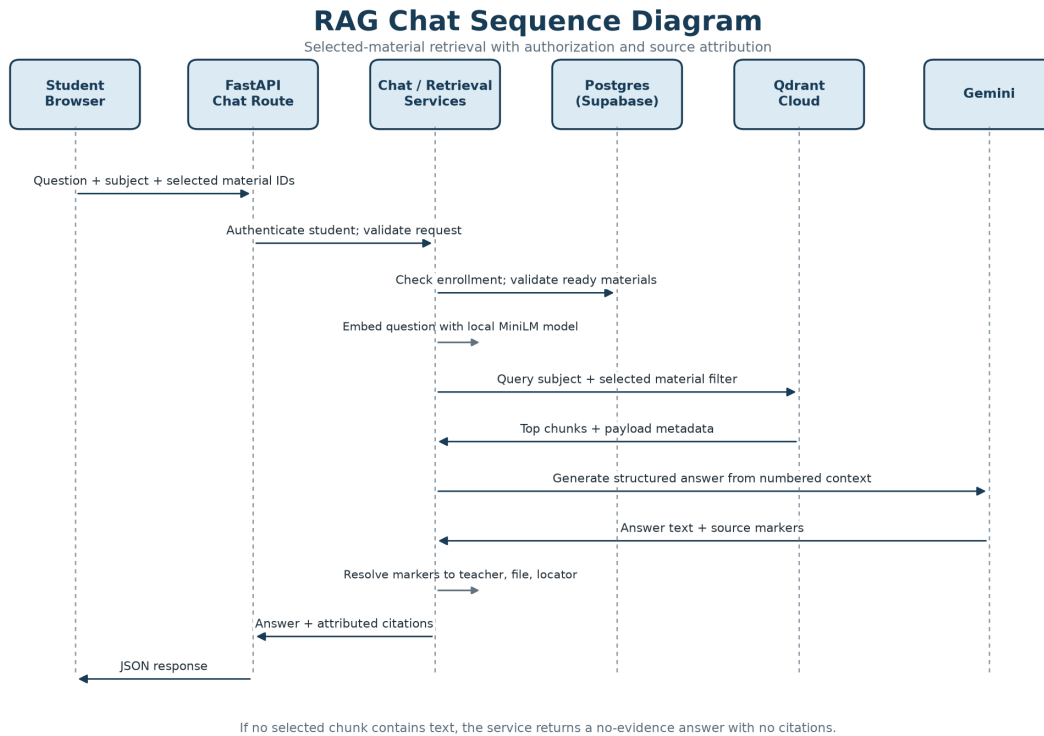


Figure 7.4: RAG chat sequence

7.5 Activity Diagram

Material ingestion begins with a teacher upload. The service validates type and size, stores the private file, creates a processing record, parses the document, normalizes text and metadata, chunks content with format-aware windows, creates embeddings, upserts points in batches, and marks the material ready. A failure marks the material failed and exposes retry; deletion removes both the relational record and its vectors. Version-safe status updates prevent stale work from reviving deleted records.

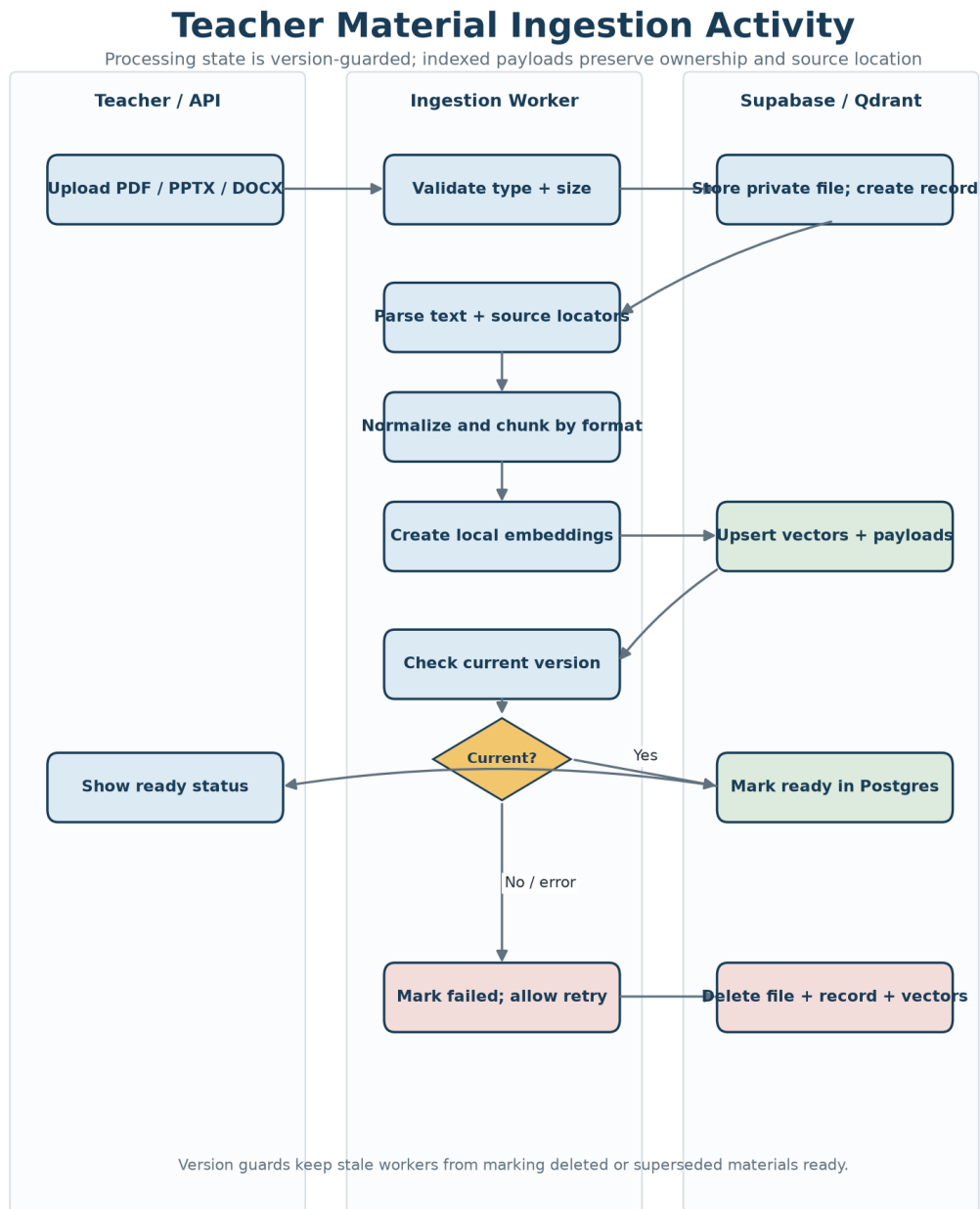


Figure 7.5: Material ingestion activity

7.6 Data Flow Diagram (DFD)

7.6.1 Level 0 DFD

At the highest level, students and teachers interact with ExamAI. ExamAI communicates with Supabase for identity, relational data, and file storage; with Qdrant for vector retrieval; and with Gemini for structured generation. The system returns study answers, assessments, feedback, decks, and analytics.

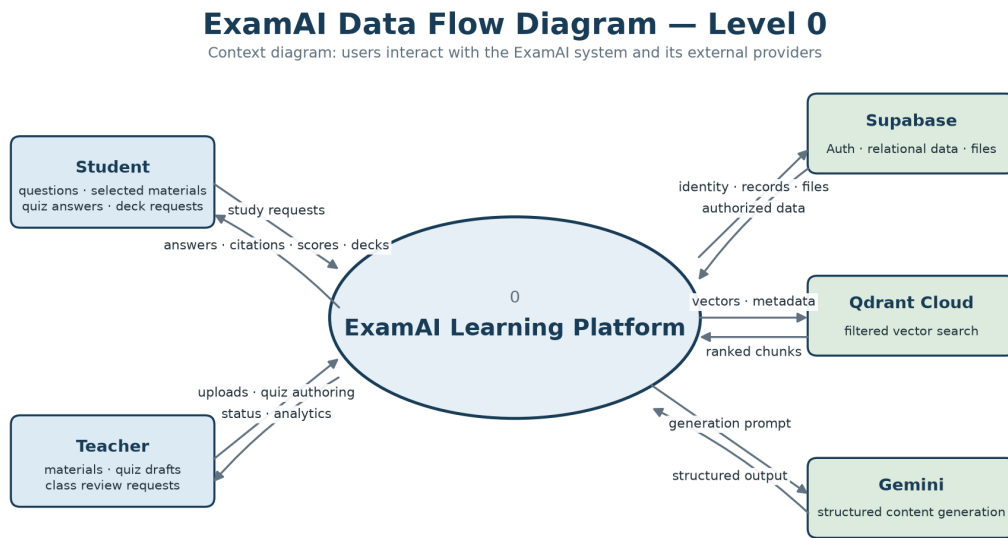


Figure 7.6.1: Level 0 DFD

7.6.2 Level 1 DFD

The level-one flow separates authentication, subject access, material ingestion, RAG chat, quiz management, flashcard generation, and analytics. Authorization is checked by the services handling subject-scoped requests. During ingestion, material, teacher, and subject identifiers are stored in each Qdrant payload; retrieval filters by subject and selected material identifiers, and citation resolution uses the returned payload metadata.

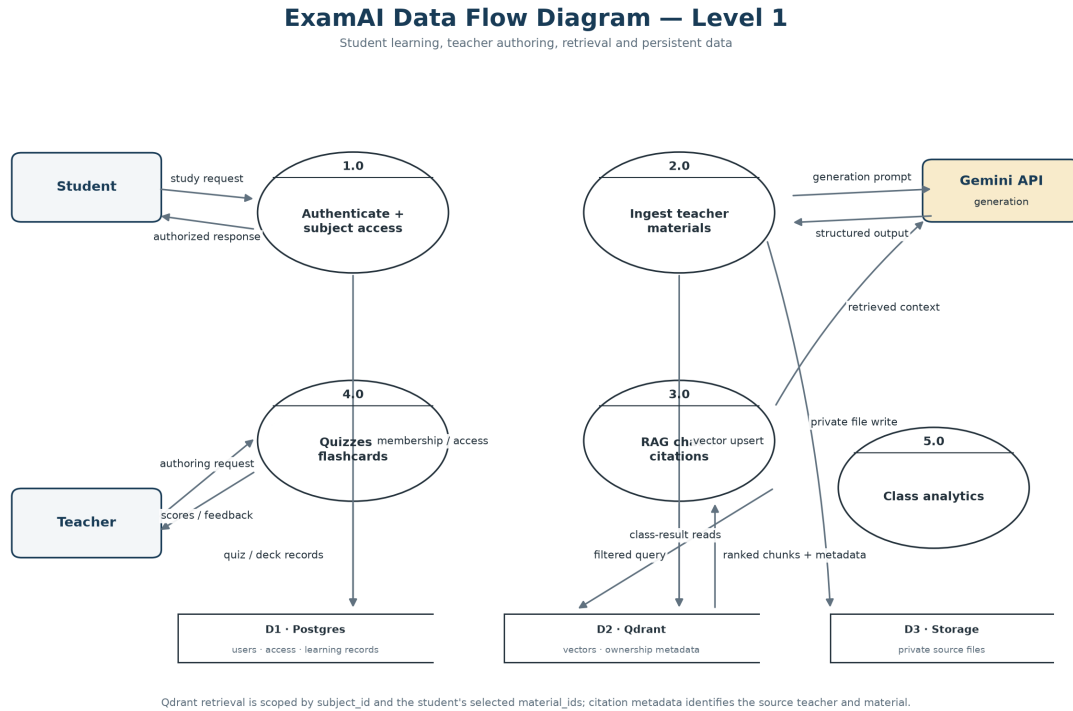


Figure 7.6.2: Level 1 DFD

7.7 Data Model

The relational model contains users, subjects, subject–teacher membership, student–subject enrollment, materials, quizzes, quiz questions, quiz attempts, flashcard decks, and flashcards. A material belongs to one teacher and one subject; a subject may have multiple teachers. A quiz is teacher-authored and subject-scoped; an attempt belongs to one student and one quiz. A flashcard deck belongs to one student and records its source material identifiers. The per-session material selection is intentionally request-time data, not a table — this keeps "adjustable per session" true at the data layer.

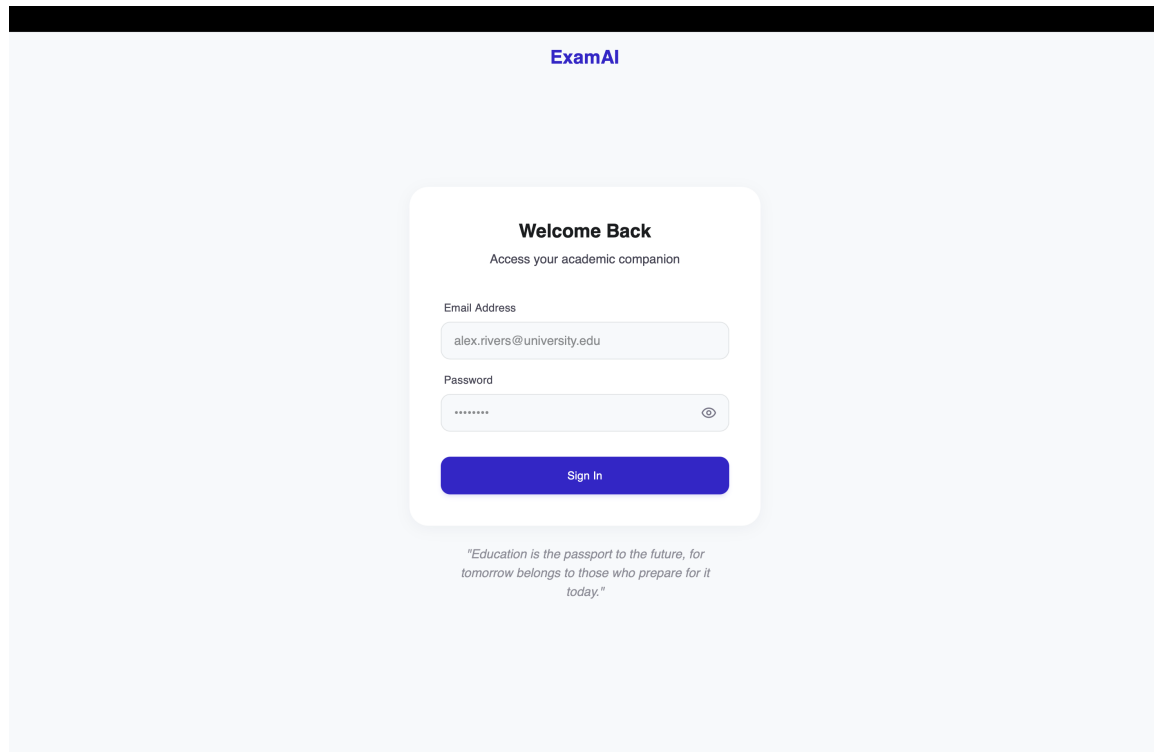
CHAPTER 8 RESULT AND DISCUSSION

- LOGIN AND ROLE-BASED ACCESS
- STUDENT DASHBOARD AND SUBJECT VIEW
- RAG CHAT WITH MATERIAL SCOPE AND CITATIONS
- QUIZ TAKING AND FEEDBACK
- FLASHCARD GENERATION AND STUDY
- TEACHER MATERIAL MANAGEMENT
- TEACHER QUIZ AUTHORING AND PUBLISHING
- TEACHER ANALYTICS AND STUDENT PROGRESS
- SECURITY AND RELIABILITY RESULTS
- OVERALL SYSTEM PERFORMANCE

ExamAI was built, tested, and verified end to end as a local prototype: every feature described in the earlier chapters works as designed against the offline test suites (82 backend, 63 frontend) plus manual end-to-end passes over a local run. This chapter walks through the finished screens in the order a user meets them, with a short discussion of each one. (Online deployment is planned but not yet carried out, so all screens below are captured from the local build running against seeded demo accounts — the names visible are seed data, not real users.)

8.1 Login and Role-Based Access

A provisioned user logs in with email and password; the backend derives the role from the stored profile and the frontend routes to the student or teacher workspace accordingly. The sign-in form offers email and password fields only — there is no signup link and no client-side role selector, so an unregistered email is rejected and an authenticated user can never reach the other role's routes.



The image shows a web interface for ExamAI. At the top, there is a black header bar. Below it, the text "ExamAI" is displayed in a blue font. The main content area has a light blue background. In the center, there is a white rounded rectangle containing the sign-in form. The form has the title "Welcome Back" and the subtitle "Access your academic companion". It includes two input fields: "Email Address" with the value "alex.rivers@university.edu" and "Password" with masked characters "*****". A blue "Sign In" button is located below the password field. At the bottom of the form, there is a quote: "Education is the passport to the future, for tomorrow belongs to those who prepare for it today."

Figure 8.1: Sign-in form — email and password only, no signup or role selector.

8.2 Student Dashboard and Subject View

The student home opens with a study hero panel leading into chat, followed by quick stats (quizzes taken, weak topics, average score) and enrolled-subject cards showing each subject's teachers and the student's progress. The demo account in Figure 8.2 shows three enrolled subjects — including multi-teacher subjects — with per-subject progress bars. The Resources view (Figure 8.3) states the ownership promise directly — "Materials approved by your teachers, grouped by course" — with browse-by-course pills, filter-by-teacher checkboxes, a Recently Added strip, and an All Materials table (name, course, date, size, owner). Only ready materials appear in this student-facing list; processing or failed items remain a teacher-side concern.

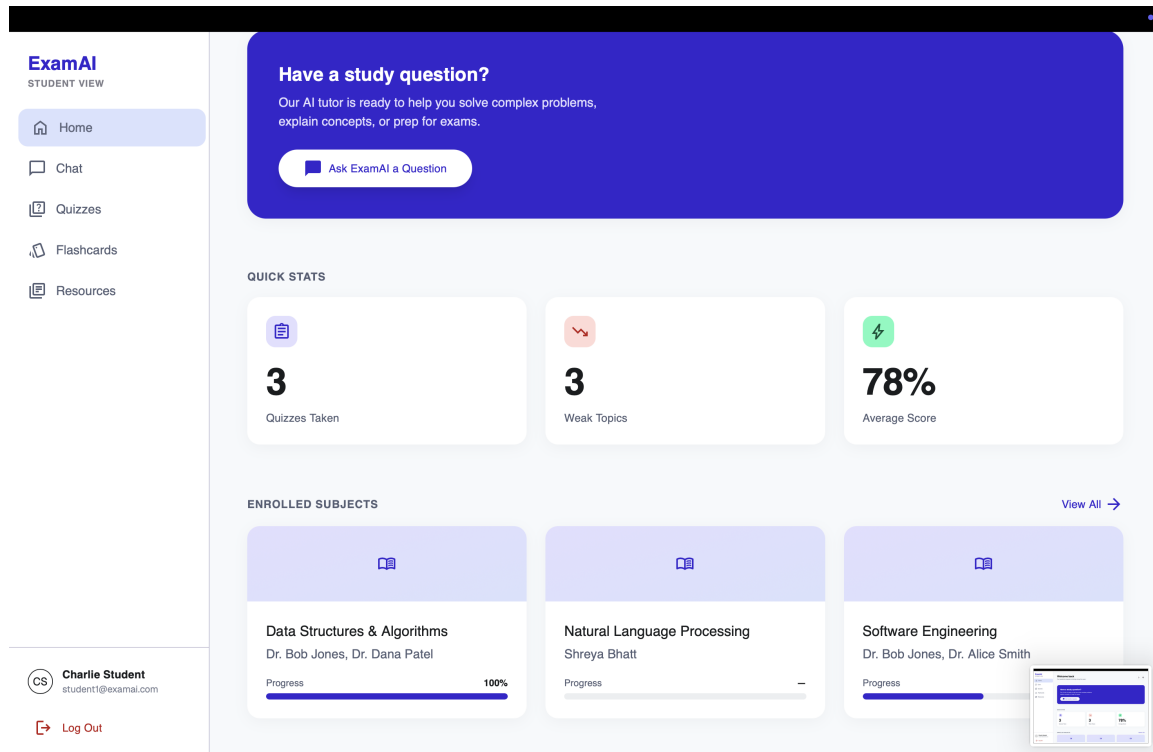


Figure 8.2: Student dashboard — study hero, quick stats, and enrolled subjects with teachers and progress.

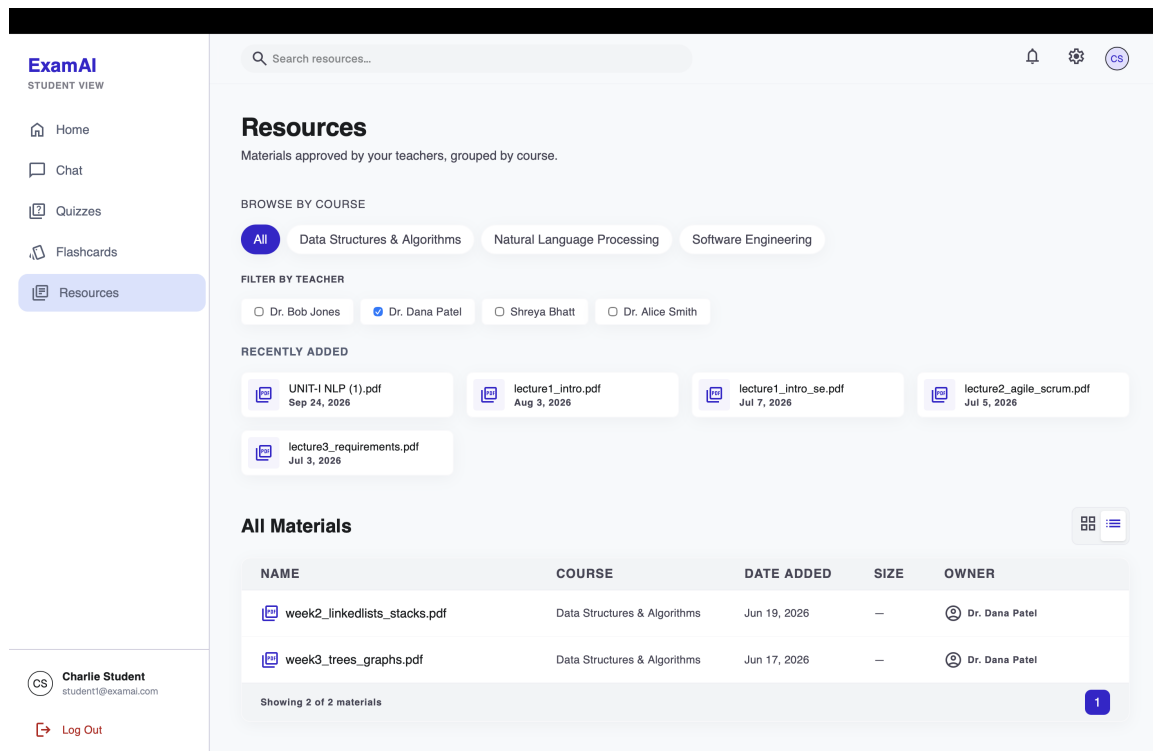


Figure 8.3: Subject resources — approved materials grouped by teacher with filter; only ready items are visible to students.

8.3 RAG Chat with Material Scope and Citations

The chat view pairs the conversation with a visible study-materials panel: the student ticks which ready materials should ground the session (grouped under the owning teacher's name), with the active count always shown. Answers render with numbered citation markers; hovering a marker reveals the teacher and filename, and a citation footer under each answer names the source explicitly. Figure 8.4 demonstrates both halves of the attribution guarantee: the in-corpus question ("What is NLP") receives a grounded answer citing the teacher's file, while the off-corpus question ("who is Donald trump") is refused with "The provided context is insufficient..." instead of hallucinated — the system answers only from approved material.

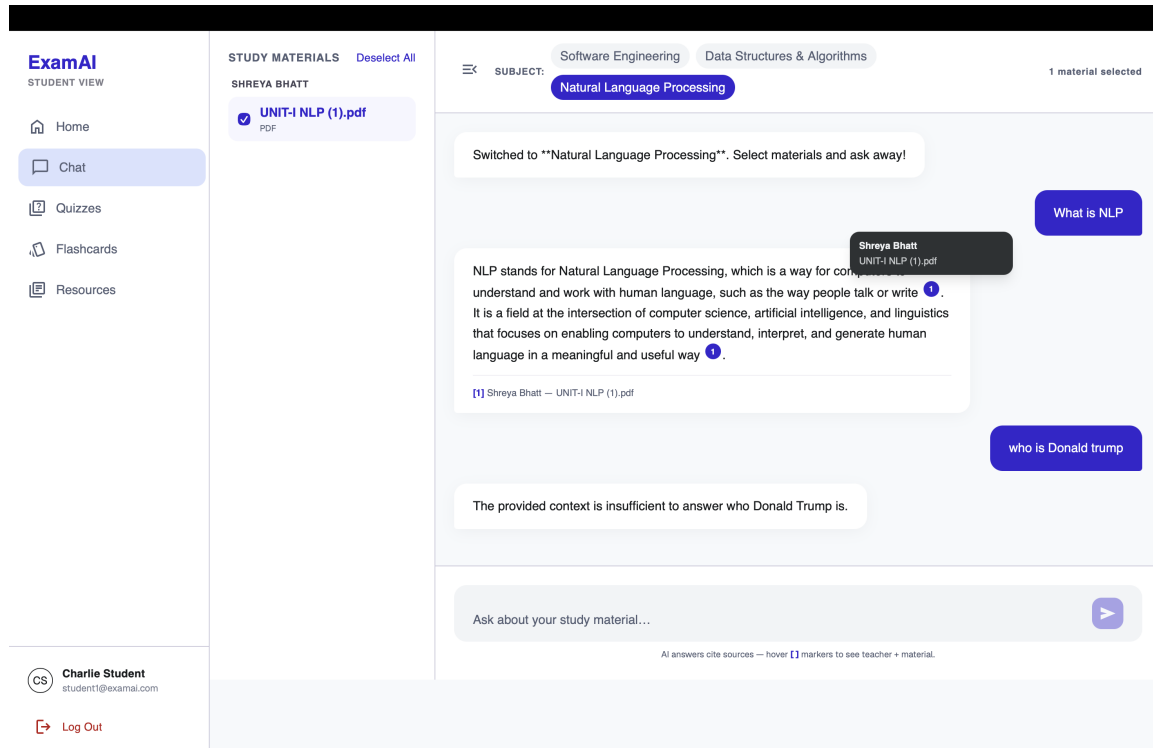


Figure 8.4: RAG chat — per-session material scope, numbered citations with teacher/file tooltip, and grounded refusal of an off-corpus question.

8.4 Quiz Taking and Feedback

The quiz list (Figure 8.5) shows four published quizzes across the student's subjects, each naming its authoring teacher: completed ones carry a Completed badge with the score and a View Results action, while the unattempted one offers Start Quiz. Taking a quiz (Figure 8.6) presents one question at a time under a visible countdown timer with Previous/Next navigation and progress dots. On submission the server grades the attempt and returns a grade card (Figure 8.7 shows a D at 50%, 3 of 6 correct) with an "Areas to Review" breakdown by topic tag (SDLC 67%, Requirements 33%), followed by a full question review (Figure 8.8) marking each response — the student's wrong pick highlighted in red against the correct choice in green, as seen in Q2 where the chosen distractor and the correct answer are shown side by side. Re-submitting an attempt is idempotent — the score is not duplicated.

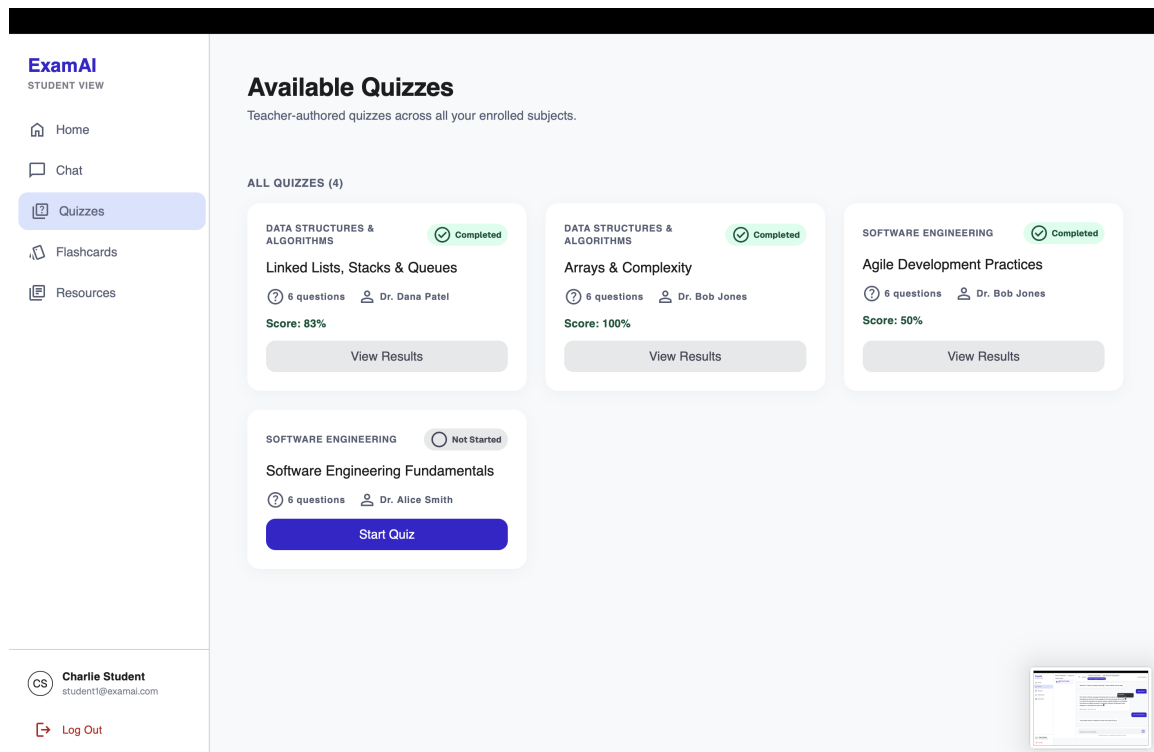


Figure 8.5: Published quiz list with completed scores.

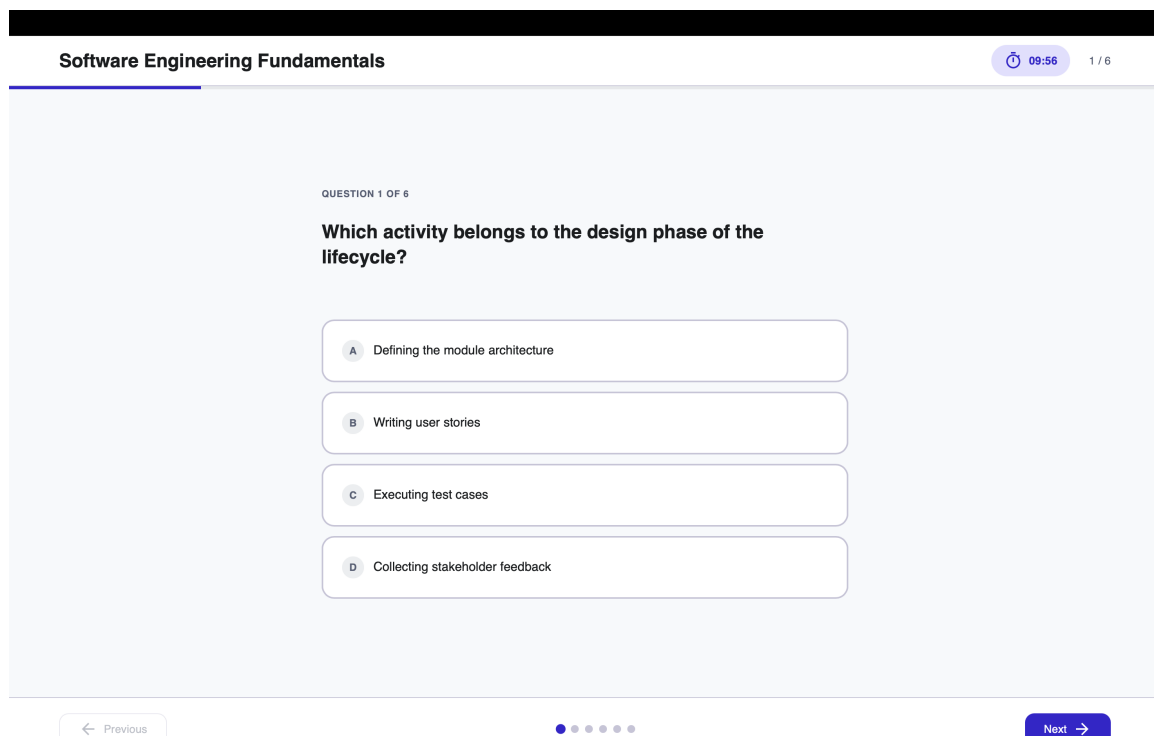


Figure 8.6: Quiz taking — one question at a time under a visible countdown with Previous/Next navigation.

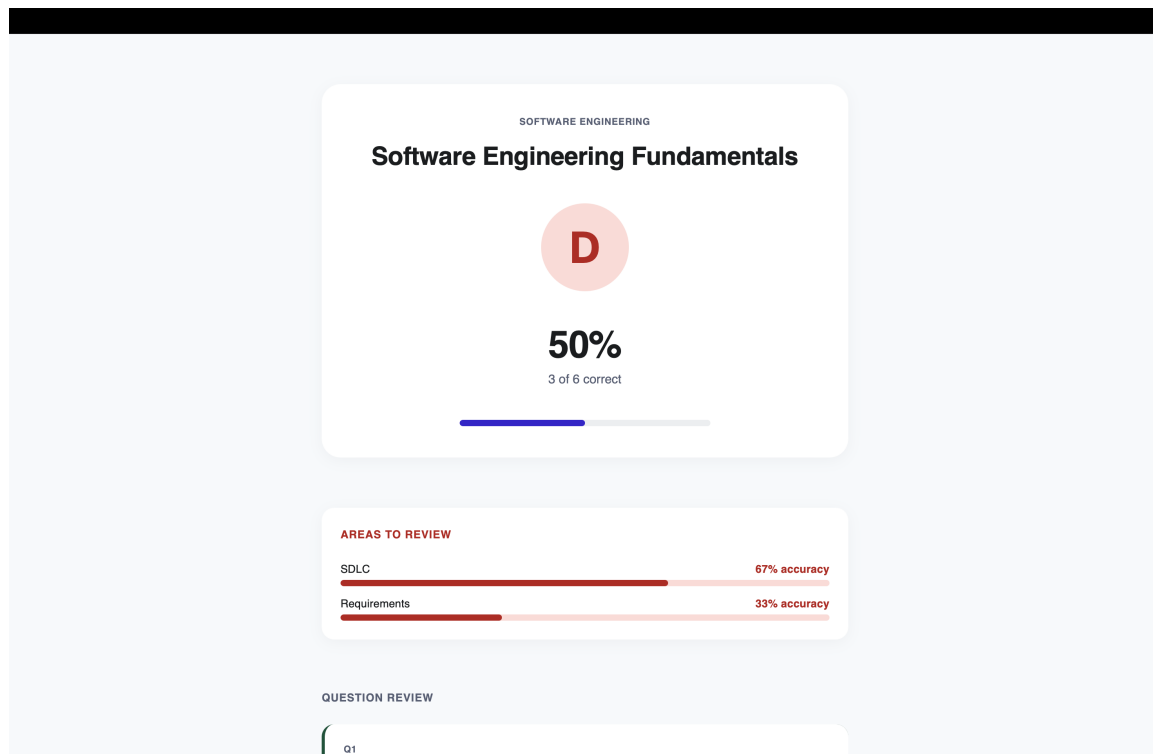


Figure 8.7: Result card — grade, score, and per-topic areas to review.

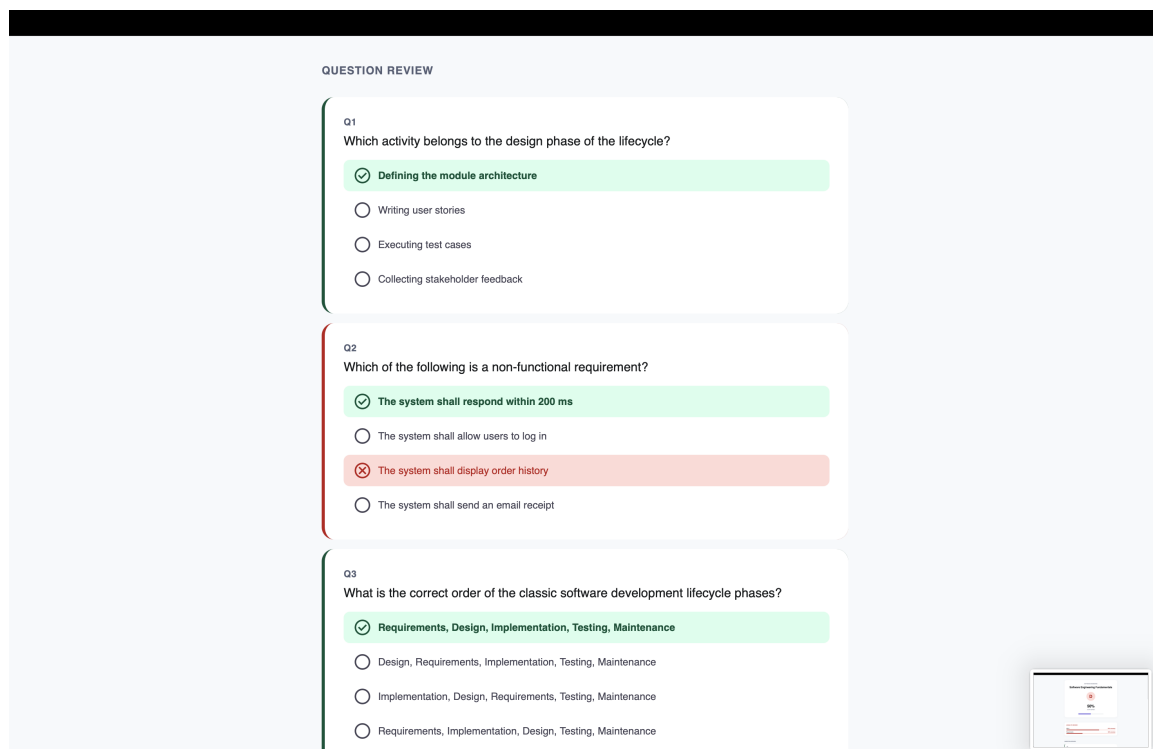


Figure 8.8: Question review — per-question correct/wrong marking against the correct choice.

8.5 Flashcard Generation and Study

From any selected material set the student generates a named deck via the Generate New Deck action; the deck list (Figure 8.9) shows two personal decks with card counts, mastered counts, and mastery bars (8 cards / 4 mastered / 50%; 10 cards / 3 mastered / 30%). Cards are studied in a flip view (Figure 8.10 shows card 1 of 8 in "Data Structures Quick Cards" with tap-to-reveal) and marked still-learning or mastered, moving the card between new, learning, and mastered states. Decks are personal to the student and record the source material identifiers used at generation, so a deck can always be traced back to what it was built from.

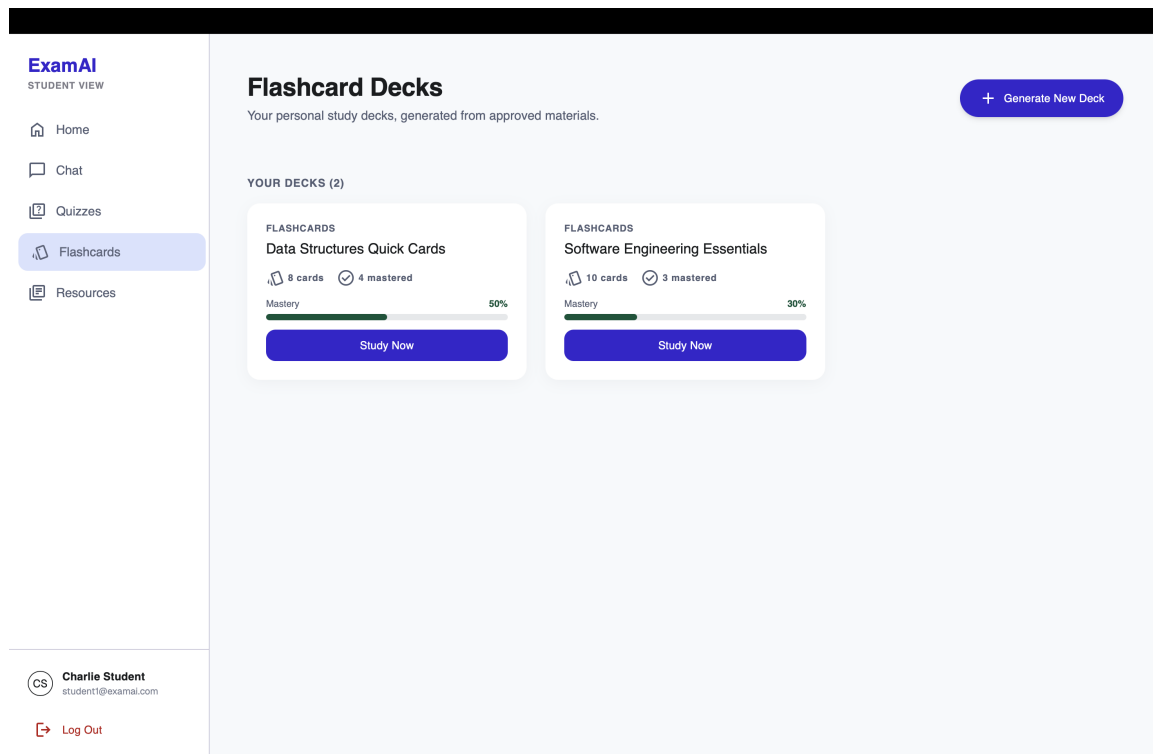


Figure 8.9: Flashcard deck list with mastery progress.

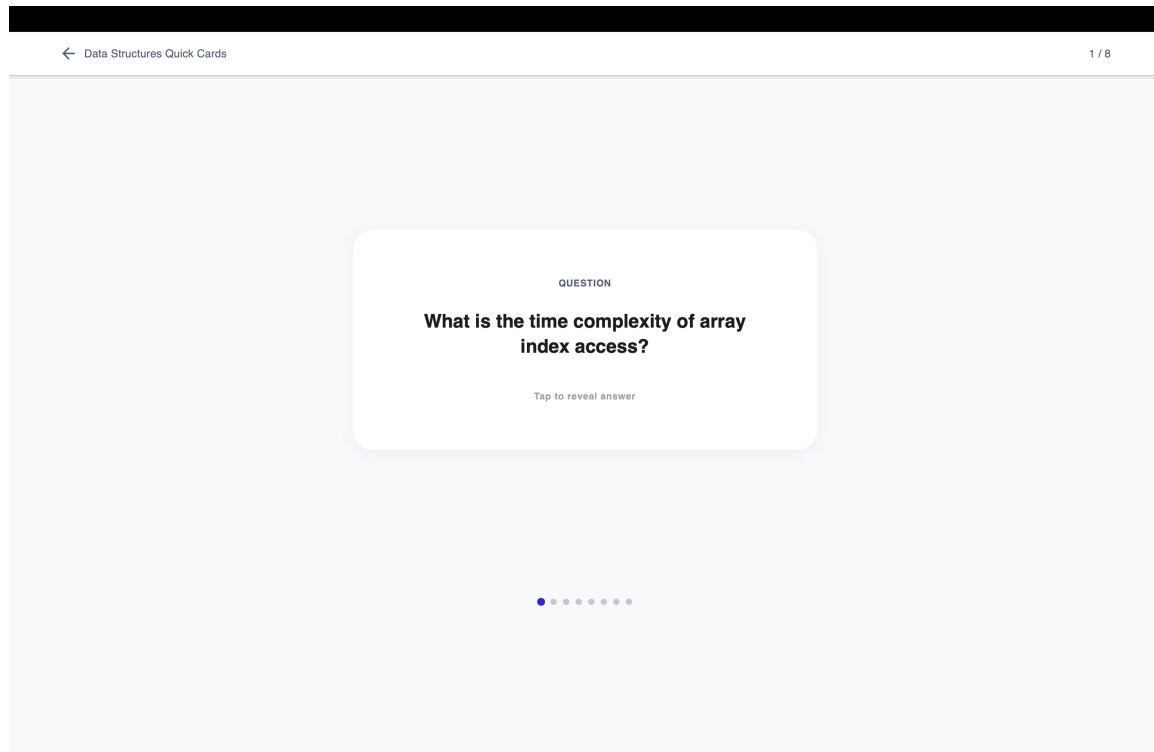


Figure 8.10: Flip-study view — question front with answer reveal and mastery controls.

8.6 Teacher Material Management

The teacher materials view ("Resources & Materials") states its sharing rule directly: "Co-teacher materials are visible but read-only." A drag-and-drop upload zone accepts PDF, PPTX, and DOCX with per-subject tabs, and the All Materials table lists file, subject, owner, status, and upload date. The Owner column distinguishes the teacher's own files ("You", editable) from a co-teacher's (visible only), and the captured rows show Ready badges — the processing and failed states with retry, plus deletion, are handled by the same pipeline (Section 7.5) and covered in testing (Chapter 6).

ExamAI
PROFESSOR VIEW

Dashboard

Materials

Quizzes

Analytics

Student Progress

Resources & Materials
Upload and manage materials. Co-teacher materials are visible but read-only.

Advanced Database Systems | Software Engineering

Click to upload materials
PDF, PPTX, DOCX supported — you'll pick the subject next

ALL MATERIALS (3)

File	Subject	Owner	Status	Uploaded
ch1_relational_algebra.pdf	Advanced Database Systems	You	Ready	Jun 27, 2026
ch2_query_optimization.pdf	Advanced Database Systems	Dr. Carol Nguyen	Ready	Jun 25, 2026
ch3_transactions_concurren...	Advanced Database Systems	Dr. Carol Nguyen	Ready	Jun 23, 2026

Dr. Alice Smith
teacher1@examai.com

Log Out

Figure 8.11: Teacher materials — upload dropzone, subject tabs, owner column separating own vs co-teacher files, Ready status badges.

8.7 Teacher Quiz Authoring and Publishing

The quiz list states the fairness rule directly: "All students in a subject take the same published quiz." Each card carries a Draft/Published badge and a Manual/AI source tag, with Edit, Publish, and delete actions — drafts expose a Publish button while published quizzes do not. Teachers author manually question by question, or request an AI-assisted draft generated from ready material; both paths produce the same question shape, and AI drafts remain drafts until reviewed and published.

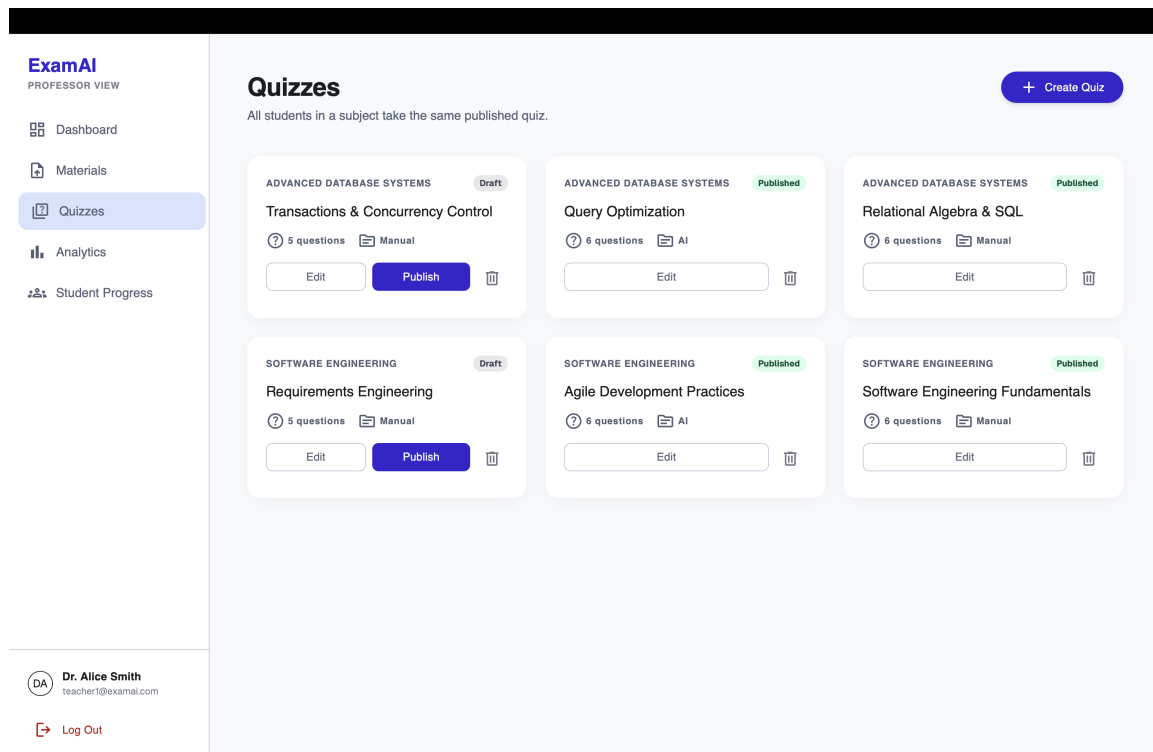


Figure 8.12: Teacher quiz list — draft/published badges, manual/AI source tags, publish flow.

8.8 Teacher Analytics and Student Progress

The analytics design deliberately separates per-quiz analysis from cross-quiz student progress. The per-quiz view (Figure 8.13, captured on a 17-student class with 16/17 completion and a 79% average) shows a question-accuracy heatmap with a Good/OK/Weak/Poor key, an A–F grade distribution, and the derived weak topic ("Indexing" at 69% class accuracy) — answering "how did this quiz go?". The student progress roster (Figure 8.14) lists every student with average score, completion bar, last-active date, and an At Risk / On Track badge, with a "7 at-risk" summary counter, name search, and subject filter on top and a drill-down chevron per row — answering "how is this student doing?". The teacher dashboard (Figure 8.15) adds an at-a-glance overview: active students, subject materials, quizzes created, and average section score (22 / 7 / 6 / 70% in the capture), with a recent-activity feed of who completed which quiz at what score, a grade-distribution summary, and an AI-insights card. Keeping these read models separate avoids forcing two different questions into one overloaded query.

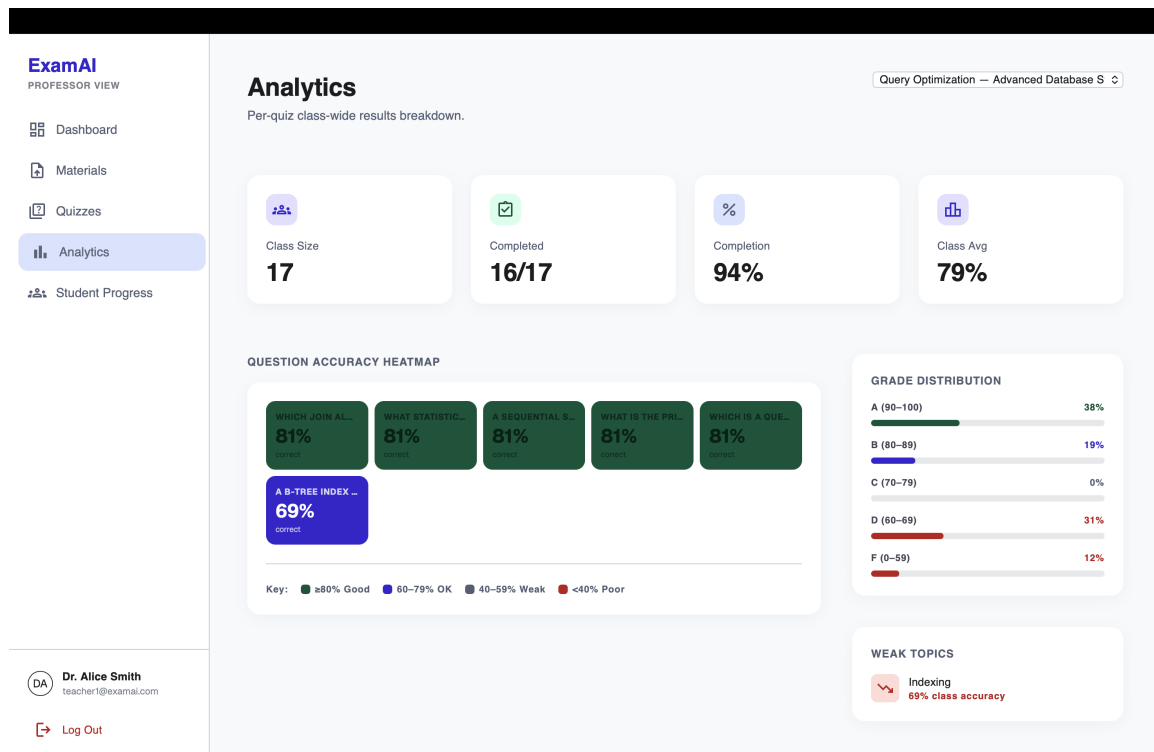


Figure 8.13: Per-quiz analytics — class stats, accuracy heatmap, grade distribution, weak topics.

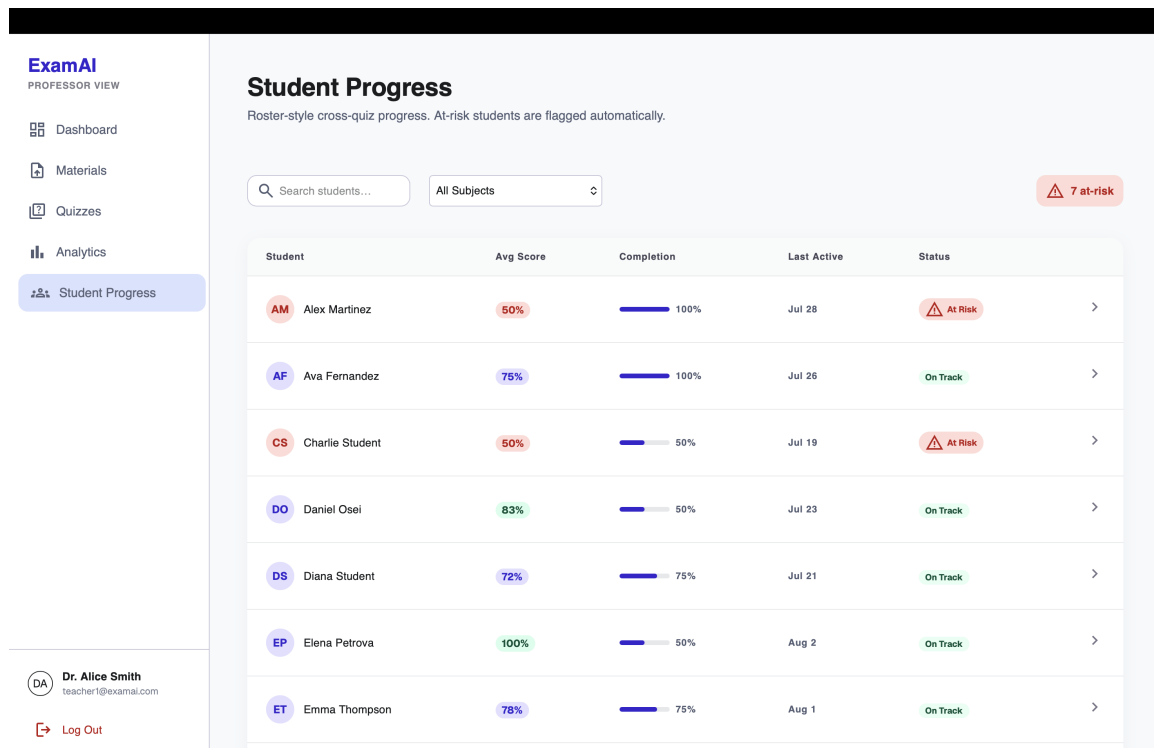


Figure 8.14: Student progress roster — averages, completion, at-risk flags with drill-down.

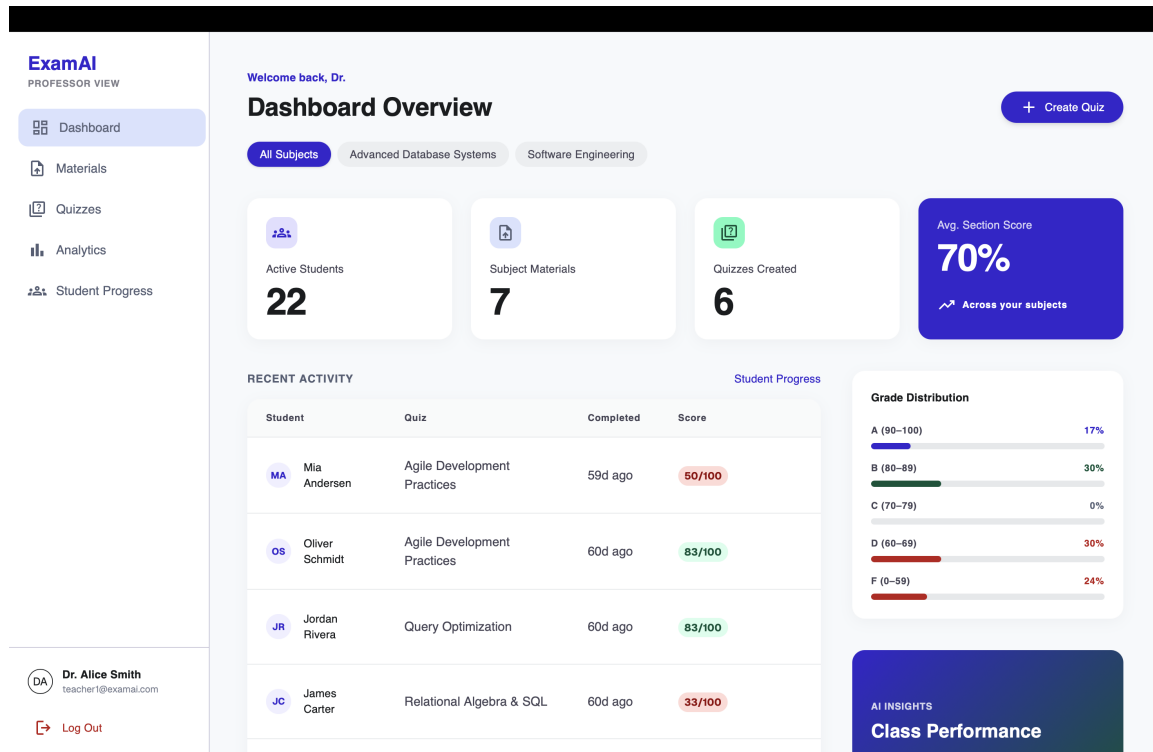


Figure 8.15: Teacher dashboard overview — activity stats, recent completions, grade distribution, AI insights.

8.9 Security and Reliability Results

Access tokens are kept in memory with a HttpOnly refresh cookie; the API client performs one silent refresh-and-replay after a 401 before redirecting to login. CORS uses an allow-list and storage paths are never serialized to clients. Authorization checks are repeated in services rather than relying on route declarations alone. The ingestion pipeline uses per-material locks, row-level guards, and version-safe updates, with blocking parser and embedding calls moved off the event loop. Structured generation uses Pydantic schemas as the output contract, retrying with the error, the bad response, and the schema on failure. These patterns improve reliability without pretending external AI services are deterministic.

8.10 Overall System Performance

The backend suite (82 tests) runs fully offline in under a second; the frontend suite (63 tests) runs under Vitest with route, store, and component coverage. Frontend performance work includes route-level code splitting, navigation prefetching, bounded safe-GET caching, concurrent request deduplication, debounced search, loading skeletons, and stale-response handling — targeting measured responsiveness while preserving correctness. The known performance caveat is dependence on hosted services (database, vector store, model API) at deployment time: latency and quota there are deployment properties, and online hosting remains planned future work (Chapter 9).

CHAPTER 9 LIMITATION AND FUTURE ENHANCEMENT

- LIMITATION
- FUTURE ENHANCEMENT

9.1 Limitation

- ExamAI is a web application only — there is no mobile app yet.
- Users are provisioned by an explicit seed operation; there is no public signup, password reset, OAuth, or administrator role.
- The generated answer is limited by the quality and coverage of uploaded material. Source attribution identifies the retrieved passage but does not guarantee the model's interpretation is correct; teachers must review AI-assisted quizzes before publication.
- Retrieval is dense-vector only; sparse/keyword signals (exact terms, codes, formulae) are not separately indexed.
- Generation features depend on the configured Gemini model, quota, and billing, which must be verified per deployment.
- There is no in-app teacher–student messaging or broadcast, and no payment or purchase flow.
- Analytics describe quiz attempts within the application; they do not claim to measure long-term learning outcomes.

9.2 Future Enhancement

- Add hybrid dense-plus-sparse retrieval and compare retrieval quality on a controlled dataset.
- Add richer source previews (page/slide thumbnails) and material version history.
- Introduce student-generated personal practice quizzes as explicit v2 scope, kept separate from shared graded assessment.
- Add institutional identity integration, administrator controls, and deployment observability (logging, metrics, backups).
- Build mobile clients against the existing REST API without backend changes.
- Run accessibility audits and extend evaluation to citation correctness, quiz quality, and study outcomes.

CHAPTER 10 CONCLUSION

- CONCLUSION

10.1 Conclusion

ExamAI brings teacher-controlled study help, shared assessment, personal revision, and class-wide analytics together into a single role-aware platform — closing the three gaps found in existing systems during the literature survey in Chapter 2: unattributed answers, incomparable assessments, and scattered teacher oversight.

The project was built end to end — from a versioned relational data model and a metadata-preserving ingestion pipeline, through filtered retrieval and schema-guarded generation, to a responsive React interface with teacher dashboards — and is verified by 82 offline backend tests and 63 frontend tests plus manual end-to-end passes. Building it strengthened practical skills in full-stack architecture, authorization design, retrieval systems, working with non-deterministic AI components, and keeping academic comparability intact while adding AI assistance.

While the current version has known limitations, listed clearly in Chapter 9 rather than left to be discovered later, the platform already delivers on its core goal: a student always knows which teacher's material — and which page — an answer came from, and a teacher always sees how the whole class performed on the same questions.

CHAPTER 11 APPENDICES

- BUSINESS MODEL
- PRODUCT DEPLOYMENT DETAIL
- API AND WEB SERVICE DETAILS

11.1 Business Model

ExamAI is currently built and presented as an academic project rather than a commercial product. If extended toward institutional use, it would naturally fit a department-level service model — the platform hosted for a department or coaching organization with teacher-managed subjects, private file storage, and usage-based AI limits. Any such model would require clear privacy, data-retention, accessibility, and service-cost policies. The current report makes no commercial deployment claim.

11.2 Product Deployment Detail

For local development, the backend runs from the `backend/` directory with Python 3.14 and its canonical virtual environment; migrations apply via Alembic, demo data seeds through the project seed command, and the FastAPI service exposes its API and documentation. The frontend starts with Vite and targets the configured API base URL.

Component	Hosting	Role
Frontend	Static build (Vite output)	Serves the React application to users
Backend	Render.com (FastAPI via Uvicorn)	Serves the REST API
Database/Auth/Storage	Supabase (managed Postgres)	Users, subjects, materials, quizzes, attempts, decks; private files
Vector search	Qdrant Cloud (single collection)	Embedded material chunks with filterable payloads
Generation	Gemini API	Structured chat, quiz, and flashcard generation

Online deployment is planned but not yet carried out; the current delivery is a verified local prototype with a defined production layout as tabulated above.

11.3 API and Web Service Details

ExamAI exposes its functionality through a RESTful API built with FastAPI. Every endpoint is documented automatically and can be explored interactively at `/docs` on the backend. Responses use a standardized envelope with request identification.

Method	Endpoint	Description
POST	<code>/api/auth/login</code>	Log in a provisioned user; role derived from profile

Method	Endpoint	Description
GET	/api/subjects	List accessible subjects for the current role
POST	/api/materials	Teacher upload of PDF/PPTX/DOCX with status tracking
POST	/api/chat	Ask a question over subject + selected material IDs
GET/POST	/api/quizzes	List / create (manual or AI-assisted) quizzes; publish flow
POST	/api/quizzes/{id}/attempts	Submit an attempt; server-side grading with weak topics
GET/POST	/api/flashcards/decks	List / generate personal decks; mastery updates
GET	/api/analytics/quiz/{id}	Per-quiz heatmap, distribution, weak topics
GET	/api/analytics/progress	Cross-quiz roster with at-risk flags

Authentication is session/token based: the access token is kept in memory on the client with a HttpOnly refresh cookie, and every subject-scoped call re-checks enrollment or membership in the service layer.

BIBLIOGRAPHY

1. Moodle Project — Moodle LMS documentation, <https://docs.moodle.org/>
2. Google — Google Classroom Help, <https://support.google.com/edu/classroom/>
3. Quizlet — Flashcards and study tools, <https://quizlet.com/>
4. OpenAI — ChatGPT (reviewed as the representative general-purpose LLM chatbot for the literature survey, Chapter 2).
5. Google — NotebookLM (reviewed as the closest retrieval-grounded study assistant, Chapter 2).
6. FastAPI Documentation — <https://fastapi.tiangolo.com/>
7. SQLAlchemy Documentation — <https://docs.sqlalchemy.org/>
8. Alembic Documentation — <https://alembic.sqlalchemy.org/>
9. React Documentation — <https://react.dev/>
10. Vite Documentation — <https://vite.dev/>
11. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
12. Qdrant Documentation — <https://qdrant.tech/documentation/>
13. Supabase Documentation — <https://supabase.com/docs>
14. Google AI for Developers — Gemini API documentation, <https://ai.google.dev/gemini-api/docs>
15. Hugging Face — Sentence Transformers documentation, <https://www.sbert.net/>